BÁO CÁO CUỐI CÙNG

Dự báo lưu lượng giao thông I-94 (chiều westbound) — trạm ATR 301

Project 20 — Bài 7: Rò rỉ dữ liệu, chia tập đúng và đánh giá trung thực

Môn học	Data Mining / Machine Learning (đề tài về rò rỉ dữ liệu & đánh giá trung thực)
Nhóm	Nhóm 20 — xem mục *Nhóm & phân công* ở §17.6 (cần người dùng điền)
Bộ dữ liệu	Metro Interstate Traffic Volume — UCI ML Repository, giấy phép CC BY 4.0
Mô hình	Ridge Regression (L2), `alpha = 0,001`, solver `lsqr`
Checkpoint	3.1 — hoàn thiện Web/API + Test + Tài liệu
Trạng thái	Đã chạy thật: 392 test pass, 0 fail; server `http://localhost:8000` chạy được

Ghi chú về nguồn số liệu. Mọi con số trong báo cáo này được lấy từ artifact do mã nguồn sinh ra: `models/model_metadata.json`, `models/run_config.json`, `models/baseline_meta.json`, `reports/figures/evaluation_results.json`, `reports/figures/experiments_results.json`, `reports/figures/alpha_tuning.json`, `data/processed/data_audit.json`. Không có con số nào được gõ tay. Khi chạy lại pipeline, báo cáo tự cập nhật. **Phân biệt thể loại nội dung** — dùng nhất quán trong toàn bộ báo cáo: - **[BẮT BUỘC]** — yêu cầu trực tiếp từ đề tài / dữ liệu / giảng viên. - **[QUYẾT ĐỊNH]** — lựa chọn của nhóm, được nêu rõ lý do. - **[HẠN CHẾ]** — điều mô hình không làm được hoặc dữ liệu không cho phép.

MỤC LỤC

• Tóm tắt điều hành
• Đặt vấn đề và mục tiêu nghiên cứu
• Dữ liệu và chất lượng dữ liệu
• Tiền xử lý và chống rò rỉ
• Chia tập theo thời gian
• Phân tích khám phá (EDA, chỉ trên TRAIN)
• Baseline và mô hình chính
• Chọn tham số alpha
• Kết quả FINAL TEST 2018
• Phân tích lỗi chi tiết
• Thí nghiệm phát triển (2012–2017)
• Ứng dụng Web / API
• Kiểm thử tự động
• Model Card
• Hạn chế và ngoài phạm vi sử dụng
• Kết luận và hướng mở rộng
• Phụ lục

1. Tóm tắt điều hành

Bài toán: ước lượng **lưu lượng giao thông theo giờ** tại trạm đo ATR 301 trên đường I-94, chiều westbound, chỉ từ thông tin lịch và thời tiết — thông tin **biết trước hoặc quan sát được tại đúng giờ cần dự báo**.

Kết quả chính trên **FINAL TEST 2018** (6.533 giờ, hoàn toàn ngoài mẫu huấn luyện):

Mô hình	MAE (xe/giờ)	RMSE (xe/giờ)	R^2
Baseline: trung bình theo `giờ × thứ`	272,90	473,13	0,9426
Ridge pipeline (alpha = 0,001)	**259,73**	**416,78**	**0,9554**
Cải thiện	**−13,17 (−4,8 %)**	−56,35 (−11,9 %)	+0,0128

Ba kết luận quan trọng nhất:

• **Trên 2018, mô hình nhỉnh hơn baseline — với điều kiện đi kèm.** Cả hai mô hình được đánh giá

trên *đúng một tập dữ liệu* 2018, đều fit trên *đúng một tập* 2012–2016. Block bootstrap theo ngày lịch cho thấy hiệu MAE **+13,16** với CI 95 % **[+3,80; +22,00]** (không chứa 0), Ridge thắng ở **8/9 tháng**. **Nhưng** trên dữ liệu dev 2012–2017 Ridge chỉ thắng ở **1/4 cửa sổ** → không được nói chung "mô hình luôn hơn baseline" (§9.3).

• **Sai số tập trung ở ngày lễ và thời tiết cực đoan.** MAE ngày lễ là 1.031,44 (n = 167) so với

239,49 ở ngày thường; MAE khi có tuyết là 524,50 so với 235,96 khi không có. Đây là giới hạn thật, không phải lỗi mã nguồn.

• **Khoảng cách thời gian làm thay đổi con số đánh giá, nhưng cơ chế không phải "nhìn thấy hàng

xóm". **Thí nghiệm 1b/1c (4 arm, chung một tập test, 5 seed cố định): đưa dữ liệu 2017 vào tập huấn luyện làm MAE giảm** 6,55 ± 0,37 **điểm, và phần giảm đó** không **do nhiều dòng hơn (ép cùng kích thước vẫn còn 6,76 ± 0,39) mà gắn với việc dữ liệu 2017 nằm ở** cùng tháng **với dòng cần dự báo (chênh 4,15 điểm so với khi chỉ lấy các tháng khác). Mô hình** không có đặc trưng lag** nên không thể *nhớ* giá trị dòng lân cận — đây là mô tả hiệu ứng, **không phải** bằng chứng nhân quả, và là lý do mọi kết luận trong báo cáo này đều dựa trên time split.

Ứng dụng web chạy tại `http://localhost:8000` với 3 màn hình và 4 endpoint; ứng dụng **chỉ nạp artifact đã đóng băng**, không huấn luyện lại, không tinh chỉnh tham số, không fit lại bộ tiền xử lý.

2. Đặt vấn đề và mục tiêu nghiên cứu

2.1 Bối cảnh

Một đơn vị vận hành giao thông muốn có một ước lượng sơ bộ về lưu lượng xe đi qua một trạm đo cố định trước khi có số đo thực tế, để hỗ trợ lập kế hoạch. Dữ liệu lịch sử cho trạm này đã có, nhưng mô hình dự báo phải đáp ứng hai ràng buộc khó nhất:

• chỉ được dùng thông tin **có thật và biết trước tại thời điểm dự báo**;

• kết luận phải **trung thực**, tức không được nhìn tương lai khi đánh giá quá khứ.

[BẮT BUỘC] Bài 7 của đề tập trung vào *data leakage* và *đánh giá trung thực*. Vì vậy phần lớn công sức của nhóm dồn vào thiết kế chia tập, tiền xử lý và bằng chứng — không phải vào việc thử nhiều mô hình phức tạp.

2.2 Câu hỏi nghiên cứu

• Đánh giá ngẫu nhiên (random split) và đánh giá trên tương lai (time split) chênh lệch bao nhiêu,

và cái nào là cái đúng?

• Mô hình có vượt được baseline theo lịch (`giờ × thứ trong tuần`) không?

• Sai số tập trung ở đâu: giờ nào trong ngày, ngày nào trong tuần, ngày lễ, hay thời tiết cực đoan?

• Chất lượng có trôi dạt (drift) theo thời gian không?

• Có thể đưa mô hình ra phục vụ mà không tạo *train-serving skew* không?

2.3 Đơn vị quan sát, đầu vào, đầu ra

Đơn vị quan sát	1 giờ tại trạm đo ATR 301
Biến mục tiêu	`traffic_volume` — **lưu lượng, đơn vị xe/giờ**
Loại bài toán	Hồi quy chuỗi thời gian
Đầu vào lúc dự báo	Ngày giờ + nhiệt độ + lượng mưa/tuyết 1 giờ + độ phủ mây + hiện tượng thời tiết
Thời điểm biết được	Tất cả đều là dữ liệu thời tiết quan sát tại giờ đó hoặc lịch công cộng biết trước

3. Dữ liệu và chất lượng dữ liệu

Nguồn: `data/processed/data_audit.json` và `reports/figures/data_quality_report.md`.

3.1 Nguồn và giấy phép

• **Metro Interstate Traffic Volume**, UCI Machine Learning Repository.

• Giấy phép: **CC BY 4.0**.

• **Trích dẫn chuẩn (bắt buộc ghi nguồn):**

Hogue, J. (2019). Metro Interstate Traffic Volume [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5X60B

• Tải bằng `py src\download_data.py`; SHA256, tên tác giả, năm, DOI và giấy phép được ghi ở `data/README.md` — hai tài liệu này dùng **cùng một** trích dẫn.

3.2 Quy mô

Mốc	Giá trị
Số dòng thô	**48.204**
Số timestamp duy nhất	40.575
Số nhóm trùng `date_time`	**5.445** (mỗi nhóm 2–6 dòng; 13.074 dòng nằm trong nhóm trùng)
Số dòng loại bởi collapse	7.629
Số dòng dùng để mô hình hoá	**40.575**
Khoảng thời gian	2012-10-02 09:00:00 → 2018-09-30 23:00:00
Số giờ *vắng mặt*	11.976 (22,79 % so với 52.551 giờ lý thuyết)

[QUYẾT ĐỊNH] Không nội suy (interpolate) các giờ vắng mặt. Một giờ không có quan sát thì không tạo ra quan sát giả; việc thiếu dữ liệu được phản ánh trong mục Hạn chế thay vì che giấu bằng thống kê học từ tập lớn.

3.3 Ba bẫy dữ liệu đã được phát hiện và xử lý

(a) Trùng `date_time` — bẫy lớn nhất

Dataset gốc có 5.445 timestamp xuất hiện 2–6 lần, mỗi giờ có thể mang **nhiều mô tả thời tiết khác nhau** (ví dụ một bản ghi "mưa nhẹ" và một bản ghi "mưa vừa").

[QUYẾT ĐỊNH] Không dùng `drop_duplicates(keep="first")`: nó phụ thuộc **thứ tự dòng trong file** (không tất định, khó tái lập) và **vứt bỏ thông tin thời tiết** của các dòng bị loại. Cách nhóm xử lý (tất định, không học thống kê từ dữ liệu):

• phép đo (`traffic_volume`, `temp`, `rain_1h`, `snow_1h`, `clouds_all`): lấy **trung vị**;

• `weather_main`: **multi-hot** — giữ mọi hiện tượng xuất hiện; nhãn đại diện là giá trị xuất hiện nhiều nhất, hoà thì lấy theo thứ tự alphabet;

• `weather_description`: nén 38 chuỗi thô về 11 nhóm gia đình bằng quy tắc keyword, lấy nhóm nghiêm trọng nhất.

Bằng chứng an toàn: `traffic_volume` và `holiday` **bất biến 100 %** trong mọi nhóm trùng (n = 5.445) — collapse không làm thay đổi mục tiêu ở bất kỳ dòng nào. `weather_main` và `weather_description` biến thiên ở lần lượt 5.349 và 5.386 nhóm — đúng lý do phải multi-hot thay vì lấy một dòng.

(b) Giá trị vô lý

Quy tắc	Số dòng	Lý do
`temp <= 0,0 K`	10	0 K là nhiệt độ tuyệt đối — quy tắc **vật lý**, không phải ngưỡng thống kê
`rain_1h = 9831,3`	1	Khớp **chính xác** một giá trị sentinel đã audit trong bản phát hành này

[QUYẾT ĐỊNH] Không dùng ngưỡng thống kê (ví dụ "trên 99,9 phân vị là sai"). Một ngưỡng suy ra từ phân bố toàn tập sẽ là học thống kê trên cả validation và test — tức rò rỉ.

Giá trị thiếu sau đó được `SimpleImputer(median)` điền **bên trong pipeline**, fit trên TRAIN.

(c) Cột `holiday` và bẫy `keep_default_na`

Cột `holiday` chứa chuỗi `"None"` (không phải ngày lễ), không phải giá trị rỗng. Nếu đọc bằng mặc định của `pandas`, `"None"` bị hiểu thành NaN và mọi dòng đều trông như ngày lễ.

[QUYẾT ĐỊNH] Đọc bằng `keep_default_na=False`. Audit xác nhận 0 dòng bị hiểu sai.

!Phân bố lưu lượng trên TRAIN

Hình 1 — Phân bố `traffic_volume` trên TRAIN. Nguồn: `src/eda.py`.

Ngoài ra, cột `holiday` gốc **chỉ ghi tên ở giờ 00:00** của ngày lễ. Suy ra cờ ngày lễ bằng cách quét các dòng khác trong cùng ngày thì vừa **không làm được** lúc dự báo (ta chỉ có *một* dòng — đó chính là train-serving skew), vừa **rò rỉ** (để biết ngày X có lễ hay không, ta đã phải "nhìn" dữ liệu của chính ngày X).

[QUYẾT ĐỊNH] Xây dựng **lịch ngày lễ tất định** trong `src/holidays.py`, tính thuần từ ngày tháng: 10 ngày lễ liên bang theo quy tắc lịch + bảng ngày khai mạc Minnesota State Fair do bang công bố. **Đối chiếu với dataset:** lịch khớp **53/53** ngày lễ mà dataset ghi nhận, bỏ sót **0** ngày, không sai tên — bằng chứng lịch tất định là đúng, không phải giả định.

Lịch ngày lễ là **thông tin công cộng biết trước**: ai cũng biết 4/7 là ngày lễ trước khi nó tới. Vì vậy dùng nó làm feature là hợp lệ, **không phải rò rỉ**.

4. Tiền xử lý và chống rò rỉ

4.1 Nguyên tắc phân loại biến đổi

Loại	Ví dụ	Fit trên	Ảnh hưởng
Quy tắc tất định	°C→K, giờ/thứ/tháng, `hour_dow`, `is_holiday`, multi-hot thời tiết, collapse median	—	Không bao giờ gây rò rỉ hay train-serving skew
Thống kê học được	`SimpleImputer(median)`, `StandardScaler`, `OneHotEncoder`	**TRAIN duy nhất**	Phải nằm trong `Pipeline`

Mọi thống kê học được nằm trong một `sklearn.pipeline.Pipeline` duy nhất, được `fit` trên tập TRAIN 2012–2016 và `transform` trên validation/test/serving. `src/train.py` chỉ gọi `fit` đúng một lần, trên `X_train`.

4.2 Bảy quy tắc chống rò rỉ của nhóm

• Mọi biến đổi trước khi tách tập là **quy tắc tất định**, không học thống kê.

• Imputer/scaler/encoder chỉ fit trên TRAIN.

• Không dùng ngưỡng hậu nghiệm rút ra từ phân bố toàn bộ tập dữ liệu.

• FINAL TEST 2018 không tham gia tuning hay model selection.

• Không `interpolate` trước khi tách tập.

• `traffic_volume` không bao giờ làm feature.

• Không trộn metric in-sample (train) với out-of-sample (val/test).

Quy tắc 4 được **cưỡng chế bằng mã nguồn**, không chỉ bằng lời hứa: `src/experiments.py` gọi `assert_no_final_test_rows()` ở mọi hàm và sẽ dừng chương trình nếu bất kỳ dòng năm 2018 nào lọt vào (`tests/test_experiments.py` — 36 test bảo vệ điều này).

5. Chia tập theo thời gian

Tập	Khoảng thời gian	Số dòng	Vai trò
TRAIN	2012-10-02 09:00:00 → 2016-12-31 23:00:00	25.329	Fit mô hình, imputer, scaler, encoder, baseline
VALIDATION	2017-01-01 00:00:00 → 2017-12-31 23:00:00	8.713	Chọn `alpha`
FINAL TEST	2018-01-01 00:00:00 → 2018-09-30 23:00:00	6.533	**Chỉ đánh giá**

Ba assert được kiểm tra mỗi lần chạy và ghi vào artifact:

• `max(train) < min(validation)` → đúng

• `max(validation) < min(test)` → đúng

• không có timestamp nào chung giữa ba tập → đúng

[QUYẾT ĐỊNH] Dùng **expanding-window time split** thay vì `train_test_split` ngẫu nhiên. Ngẫu nhiên hoá sẽ đưa các giờ của năm 2018 vào tập huấn luyện, khiến mô hình "nhìn thấy" hàng xóm của chính dòng cần dự báo — đây chính là dạng rò rỉ mà đề tài muốn chỉ ra. Mục 11.1 đo lại mức độ lạc quan do random split, có tách riêng kích thước tập huấn luyện và khoảng cách thời gian.

[HẠN CHẾ] FINAL TEST chỉ kéo dài tới **30/09/2018** — không có dữ liệu tháng 10–12/2018, và ba tháng cuối năm lại là những tháng khó nhất. Định lượng hệ quả bằng chính validation: chấm cùng mô hình trên Jan–Sep 2017 cho MAE 250,96, tức **thấp hơn cả năm 21,16** — nghĩa là số 259,73 của FINAL TEST được đo trên **phần dễ hơn** của năm. Phép so sánh với baseline vẫn công bằng (cùng thiếu quý IV). Chi tiết ở §10.9.

6. Phân tích khám phá (EDA, chỉ trên TRAIN)

Nguồn: `reports/figures/eda_train_only.md`.

6.1 Phân bố mục tiêu (TRAIN)

Thống kê	Giá trị
Số quan sát	25.329
Min / Max	0 / 7.260
Mean / Std	3.252,51 / 1.987,14
Median	3.339
p05 / p95	336 / 6.199
Số giờ có lưu lượng = 0	2

6.2 Hình dạng theo giờ × thứ — cấu trúc mạnh nhất của bài toán

Lưu lượng trung bình theo giờ và thứ (TRAIN) cho thấy rõ:

• **Hai đỉnh sáng**: ~07:00–08:00 (ngày làm việc) và ~16:00–17:00;

• **Đáy ban đêm** 02:00–03:00 (khoảng 280–400 xe/giờ ngày làm việc);

• **Cuối tuần hoàn toàn khác hình dạng**: thứ Bảy–Chủ nhật đường cong phẳng, đỉnh dịch sang khoảng

15:00–17:00 và cao hơn nhiều vào ban đêm (ví dụ 00:00 Chủ nhật 1.335 so với 617 thứ Hai).

!Lưu lượng trung bình theo giờ và thứ trong tuần

Hình 2 — Lưu lượng trung bình theo giờ × thứ, TRAIN. Nguồn: `src/eda.py`. Hai đỉnh sáng và chiều, và hình dạng cuối tuần khác hẳn ngày làm việc, đều thấy rõ ở đây.

Đây chính là cấu trúc mà **baseline khai thác** và là lý do `hour_dow` (tương tác giờ × thứ) là feature chính của mô hình.

6.3 Ảnh hưởng của tháng và ngày lễ

• Trung bình theo tháng nằm trong khoảng ~2.800–3.800, thấp nhất ở tháng 12 và 1, cao hơn vào

tháng 6–8. **[QUYẾT ĐỊNH]** đưa `month` vào nhóm categorical. !Lưu lượng trung bình theo tháng và năm

Hình 3 — Lưu lượng trung bình theo tháng × năm, TRAIN. Nguồn: `src/eda.py`. Các ô trống ở 2012 (chỉ có từ tháng 10) và 2015 (thiếu quý I–II) là do thiếu dữ liệu, không phải do lọc.

• Ngày lễ làm đường cong lưu lượng thay đổi mạnh → cờ `is_holiday` là feature **bắt buộc**.

!Ảnh hưởng của ngày lễ

Hình 4 — Ảnh hưởng của ngày lễ, TRAIN. Nguồn: `src/eda.py`.

7. Baseline và mô hình chính

7.1 Baseline — bắt buộc theo đề

[BẮT BUỘC] Đề yêu cầu so sánh mô hình với một baseline hợp lý.

[QUYẾT ĐỊNH] Baseline là **trung bình lưu lượng theo `giờ × thứ trong tuần`, fit** chỉ trên TRAIN** (25.329 dòng), lưu ở `models/baseline_table.csv`. Có `global_fallback = 3.252,51` cho ô trống. Baseline dùng **đúng tập huấn luyện** như Ridge → phép so sánh trên test là công bằng.

Vì sao baseline này mạnh: nó khai thác đúng cấu trúc mạnh nhất đã tìm thấy ở EDA (§6.2), và không dùng thời tiết. Nếu Ridge chỉ bằng hoặc thua baseline thì mô hình không có giá trị.

7.2 Mô hình chính

[QUYẾT ĐỊNH] Chọn **Ridge Regression (L2)** trong `sklearn.pipeline.Pipeline`:

```
ColumnTransformer
├── cat (4 cột) : OneHotEncoder(handle_unknown="ignore", sparse_output=False)
├── num (5 cột) : SimpleImputer(strategy="median") → StandardScaler()
└── bin (12 cột) : passthrough
↓
Ridge(alpha, solver="lsqr")
```

Vì sao Ridge: đề tài thiên về minh hoạ phương pháp đánh giá, không phải săn điểm. Ridge tuyến tính, dễ giải thích, chịu được đa cộng tuyến tốt, và cho phép so sánh ý nghĩa từng nhóm feature. Một mô hình phức tạp hơn (gradient boosting) sẽ cho MAE tốt hơn nhưng làm sai lệch trọng tâm của đề bài, và khó trả lời "vì sao".

`solver="lsqr"` vì ma trận sau one-hot khá rộng (217 cột) nhưng mẫu chỉ 25.329; `lsqr` ổn định và nhanh, tránh vấn đề lập ma trận phân rã.

7.2.1 Danh mục feature (217 cột sau tiền xử lý)

Nhóm	Số cột	Tên
Categorical	4	`hour_dow`, `month`, `weather_main_mode`, `weather_family`
Numeric	5	`temp`, `rain_1h`, `snow_1h`, `clouds_all`, `weather_severity`
Binary	12	`is_holiday`, `wm_clear` … `wm_thunderstorm` (11 cột multi-hot)

Ghi chú thiết kế:

• `hour_dow` = `giờ × thứ` — đúng độ chi tiết baseline dùng, để Ridge học được cùng effect đó **cộng thêm** thời tiết và tháng.

• `temp` lưu ở **Kelvin** (nhiệt độ tuyệt đối) vì đó là cách dataset lưu. API nhận **°C** và tự quy đổi.

• `weather_severity` là thang mức độ **do analyst định nghĩa** (0 quang → 4 giông/bão), dùng để gộp nhiều mô tả thời tiết của cùng một giờ thành một số.

• 11 cột `wm_*` là **multi-hot**: một giờ có thể vừa mưa vừa tuyết vừa giông.

7.2.2 Thống kê imputer/scaler đã học (lưu trong `model_metadata.json`)

Cột	Median (imputer)	Mean (scaler)
`temp`	282,08	280,9479
`rain_1h`	0,0	0,1054
`snow_1h`	0,0	0,0002
`clouds_all`	40,0	45,433
`weather_severity`	1,0	1,1751

Đây là **bằng chứng máy đọc được** rằng bộ tiền xử lý đã học trên TRAIN (ví dụ mean `clouds_all` = 45,43 là trung bình của TRAIN, không phải của toàn bộ 48.204 dòng).

8. Chọn tham số alpha

[BẮT BUỘC] Tiêu chí: **MAE trên VALIDATION 2017**. Test 2018 không được mở ra ở bước này.

alpha	MAE (val)	RMSE (val)	R^2
0,001	**272,12**	421,46	0,9548
0,003	272,13	421,46	0,9548
0,01	272,14	421,46	0,9548
0,03	272,19	421,47	0,9548
0,1	272,35	421,50	0,9548
0,3	272,81	421,59	0,9548
1,0	274,58	422,04	0,9547
3,0	280,77	424,37	0,9542
10,0	312,13	442,39	0,9502
30,0	429,00	539,15	0,9260
100,0	751,03	882,70	0,8017
300,0	1.146,97	1.332,33	0,5483
1.000,0	1.461,15	1.688,71	0,2744

Kết quả: `alpha = 0,001`.

Diễn giải: Ở vùng alpha nhỏ hiệu năng gần như nhau — dữ liệu không bị quá nhiễu nên hiệu chuẩn không cần mạnh. Từ alpha ≥ 10 mô hình bị co quá nặng và MAE tăng vọt (alpha = 30 → 429,00; alpha = 1.000 → 1.461,15). Chọn biên nhỏ nhất là lựa chọn ít giả định nhất, và nó cũng cho MAE tốt nhất trên validation.

MAE trên VALIDATION của Ridge: 272,12 so với **baseline 278,66**.

8.1 Độ nhạy cảm của alpha — alpha tối ưu có nằm mép lưới không?

*Sinh bởi `src/experiments.py` (mục 6) → `reports/figures/alpha_sensitivity.json`. Chạy **chỉ trên VALIDATION 2017**.*

`alpha = 0,001` là phần tử **nhỏ nhất** của lưới gốc, tức nó nằm ngay mép lưới — cần kiểm tra xem đó là lựa chọn hợp lý hay chỉ vì lưới bị hẹp. Vì vậy nhóm chạy lại với **lưới mở rộng**, thêm cả `alpha = 0` (tức **OLS** — hồi quy tuyến tính

không hiệu chuẩn):

alpha	0 (OLS)	0,000001	0,0001	0,001 (đóng băng)	0,01	0,1	1,0	10,0	30,0	1.000,0
MAE (val)	272,05	272,12	272,12	**272,12**	272,14	272,35	274,58	312,13	429,00	1.461,15
Chênh so với đóng băng	**−0,07**	+0,00	+0,00	+0,00	+0,02	+0,23	+2,46	+40,01	+156,88	+1.189,03

(Bảng đầy đủ 17 giá trị nằm trong `alpha_sensitivity.json`.)

[QUYẾT ĐỊNH] Kết luận — và đây là một phát hiện trung thực, không phải điều đẹp:

> Trên toàn vùng `alpha ∈ [0; 0,01]`, MAE validation chỉ dao động trong **272,05 – 272,14**, tức **chênh nhau chỉ 0,09 xe/giờ**. `alpha = 0` (OLS) cho MAE 272,05 — chỉ tốt hơn alpha đã đóng băng **0,07**. Nói cách khác: **Ridge với alpha rất nhỏ gần như đúng bằng OLS, và hiệu chuẩn L2 gần như không cải thiện gì trên dữ liệu này.**

Vì sao? (1) Dữ liệu không đủ nhiễu để cần co hệ số. (2) Số mẫu (25.329) lớn hơn nhiều so với số đặc trưng (217), nên hệ thống phương trình vốn đã ổn định và hiệu chuẩn không thêm được gì. (3) Đây cũng là lý do chọn Ridge thay vì OLS thuần **không** phải để tăng độ chính xác, mà để giữ một siêu tham số có thể kiểm soát trong trường hợp tương lai cần hiệu chuẩn mạnh hơn.

Về vị trí mép lưới: `alpha = 0` mới là giá trị tốt nhất trên lưới mở rộng, nhưng **chỉ hơn 0,07 xe/giờ** — dưới ngưỡng "cải thiện có ý nghĩa" mà nhóm đặt ra là 0,5 xe/giờ. **Không có alpha nào** trong lưới mở rộng cải thiện có ý nghĩa so với alpha đã đóng băng. Vậy việc alpha nằm mép lưới gốc **không phải** hệ quả của lưới hẹp.

[QUYẾT ĐỊNH] Vì sao KHÔNG đổi alpha đã đóng băng: FINAL TEST 2018 **đã được xem**. Chọn lại alpha bây giờ — dù chỉ cải thiện 0,07 — là **test-informed model selection**, đúng thứ mà toàn bộ phương pháp của đồ án này cảnh báo. Muốn dùng `alpha = 0` thì phải đánh giá lại trên **một holdout mới**.

9. Kết quả FINAL TEST 2018

9.1 Bảng kết quả chính

Mô hình	MAE	RMSE	R^2	n
Baseline (`giờ × thứ`)	272,90	473,13	0,9426	6.533
Ridge pipeline (alpha = 0,001)	**259,73**	**416,78**	**0,9554**	6.533
Cải thiện	**−13,17 (−4,8 %)**	−56,35 (−11,9 %)	+0,0128	—

[BẮT BUỘC] Đây là kết luận chính thức của báo cáo.

> **Đây là số của RAW MODEL** (Ridge trả về trực tiếp, không hậu xử lý). Tầng phục vụ có thêm một bước chiếu về sàn — `DEPLOYED PREDICTOR = max(0, ·)` □ Ridge — được báo riêng ở §12.5 và **không** phải cùng một model metric. Nhóm **không** sửa bảng này cho khớp API.

9.2 Vì sao phép so sánh này đáng tin

Điều kiện	Baseline	Ridge
Tập fit	2012–2016 (25.329 dòng)	2012–2016 (25.329 dòng)
Tập đánh giá	2018-01-01 → 2018-09-30 (6.533 dòng)	**cùng**
Có dùng năm 2018 để lựa chọn không?	Không	Không

Hai mô hình được đánh giá trên **cùng một tập dữ liệu**, xây trên **cùng một tập huấn luyện**, và **không mô hình nào được chọn bằng cách nhìn năm 2018**.

9.3 Bất định của phép so sánh — kết luận có điều kiện

Nguồn: `reports/figures/uncertainty_audit.md` và `.json` (sinh bởi `src/uncertainty_audit.py`, chạy **sau** `src/evaluate.py`, chỉ đo — không quyết định gì).

Block bootstrap **cặp theo khối ngày lịch** (4.000 lần, seed cố định). Vì sao theo ngày: các giờ trong cùng một ngày lịch không độc lập (cùng thời tiết, cùng ngày làm việc), nên lấy mẫu lại từng dòng sẽ cho khoảng tin cậy **hẹp hơn thực tế**. Quy ước hiệu: **baseline − Ridge**, dương = Ridge tốt hơn.

Chỉ số	Ước lượng	CI 95 %	% lần lấy mẫu Ridge thắng	CI có chứa 0?
Hiệu MAE	**+13,16**	**[+3,80; +22,00]**	**99,58 %**	**không**
Hiệu RMSE	**+56,36**	**[+31,07; +80,63]**	**100,00 %**	**không**

Theo từng tháng của 2018: Ridge thắng ở **8/9 tháng**; thua rõ nhất ở **tháng 8** (hiệu MAE **−18,77**).

Nhưng hiệu ứng này KHÔNG nhất quán qua các năm

Cùng phép đo, áp dụng cho các cửa sổ chỉ dùng dữ liệu dev 2012–2017:

Cửa sổ	n	MAE baseline	MAE Ridge	Hiệu MAE	CI 95 %	% Ridge thắng
pseudo-test 2016–2017 (train ≤ 2015)	16.551	294,95	306,48	**−11,53**	[−19,72; −3,79]	0,18 %
fold 1 — test 2015	3.593	282,86	325,98	**−43,13**	[−60,91; −25,92]	0,00 %
fold 2 — test 2016	7.838	317,29	343,77	**−26,48**	[−38,32; −14,72]	0,00 %
fold 3 — test 2017	8.713	278,66	272,12	+6,54	[−4,50; +16,50]	88,75 %

[QUYẾT ĐỊNH] Kết luận viết đúng theo số liệu, không nói quá:

> Trên **FINAL TEST 2018**, Ridge vượt baseline **13,16 MAE** (CI 95 % [+3,80; +22,00], **không chứa 0**) và thắng ở **8/9 tháng**. Tuy nhiên trên dữ liệu dev 2012–2017, Ridge chỉ thắng ở **1/4 cửa sổ** (pseudo-test 2016–2017 và 3 fold rolling-origin), và ở 3 cửa sổ kia khoảng tin cậy **nằm hẳn về phía baseline**. Chiều ưu thế **không nhất quán** giữa các cửa sổ.

Vì sao lại thế? Các cửa sổ mà Ridge thua đều có **tập huấn luyện rất thưa** (2014 kết thúc 08/08, 2015 bắt đầu 11/06) — Ridge cần dữ liệu đủ dày để học mức lưu lượng, trong khi baseline (trung bình theo `giờ × thứ`) không cần học mức nào. Trên FINAL TEST, tập huấn luyện 2012–2016 là bản đầy đủ nhất của dự án, nên đó là nơi Ridge có cơ hội thể hiện.

Điều kiện bắt buộc khi trích dẫn kết luận "Ridge vượt baseline"

Chỉ nói về **FINAL TEST 2018 (01/01 – 30/09)**, không suy rộng ra năm khác; phải là so sánh trên **cùng một tập đánh giá** và **cùng một tập huấn luyện 2012–2016**; metric là **RAW MODEL** (`DEPLOYED PREDICTOR` được báo riêng ở §12.5); và **không** được dùng như tuyên bố tổng quát rằng "mô hình luôn hơn baseline" — số liệu 2012–2017 nói ngược lại ở 3/4 cửa sổ.

9.4 So với validation — dấu hiệu tích cực

Tập	MAE	RMSE	R^2	n
VALIDATION 2017 (dùng để chọn alpha)	272,12	421,46	0,9548	8.713
FINAL TEST 2018	259,73	416,78	0,9554	6.533

MAE và RMSE trên FINAL TEST **thấp hơn** validation, R^2 cao hơn. Nghĩa là hiệu năng **không suy giảm** khi dữ liệu dài thêm một năm — dấu hiệu mô hình không bị overfit theo thời gian.

Cần thận trọng khi diễn giải: có thể năm 2017 đơn giản hơn 2018, nên chưa thể kết luận "mô hình đang tiến bộ". Đây cũng là kết luận mà thí nghiệm rolling-origin (§11.4) nêu thẳng.

9.5 Tính trung thực khi nói về FINAL TEST 2018

Phát biểu chính xác (dùng nguyên văn trong bảo vệ):

> **Năm 2018 không tham gia hyperparameter tuning hoặc model selection. Pipeline và serving policy được đóng băng từ dữ liệu 2012–2017. Kết quả 2018 không được dùng để tiếp tục tối ưu mô hình.**

Cụ thể hóa từng vế:

• **Không tuning / model selection trên 2018.** Toàn bộ lựa chọn về tiền xử lý, đặc trưng, mô hình

và tham số `alpha` được chốt trên 2012–2017; `alpha` chọn theo MAE trên VALIDATION 2017 (§8).

• **Pipeline được đóng băng trước khi 2018 được mở ra.** Thứ tự chạy bắt buộc là

`train` → `experiments` → `freeze_serving_policy` → `evaluate` → `postprocess_audit` (§17.2).

• **Serving policy cũng được đóng băng từ 2012–2017.** `src/freeze_serving_policy.py` chỉ đọc

TRAIN + VALIDATION; `src/postprocess_audit.py` và `src/uncertainty_audit.py` chỉ **đo** hậu quả trên 2018 và **từ chối chạy** nếu policy chưa đóng băng. Vì vậy policy `max(0,·)` không phải test-informed postprocessing (§12.5).

• **Kết quả 2018 không quay ngược lại điều chỉnh mô hình.** Sau khi đọc số 2018, nhóm không sửa

feature, không sửa `alpha`, không đổi mô hình, và không sửa lại gói đánh giá để khớp API (§12.5.8).

Phát biểu này cố ý **không** khẳng định 2018 chỉ được chạy đúng một lần — nhóm không đưa ra tuyên bố không thể chứng minh. Điều cần chứng minh là **không có vòng lặp tối ưu nào đi qua 2018**, và điều đó đã được cưỡng chế bằng `assert_no_final_test_rows()` trong toàn bộ mã nguồn thí nghiệm phát triển.

Thứ tự thực tế (nói thẳng). Thứ tự lệnh `train → experiments → freeze_serving_policy → evaluate` ở §17.2 là thứ tự **tái lập**, không phải nhật ký thời gian thật. Trong thực tế, FINAL TEST 2018 đã được mở **một lần** trước các phân tích hậu nghiệm — giờ đêm, log-target, độ nhạy `alpha`, lag (`docs/project-log.md` và `reports/figures/experiments_report.md` ghi rõ các phân tích này là "khám phá hậu nghiệm"). Các phân tích đó **không** làm thay đổi mô hình đã đóng băng, không đổi `alpha`, không đổi đặc trưng và không đổi serving policy. Điều bảo đảm được — và là điều có ý nghĩa học thuật — là **không có vòng lặp tối ưu nào đi qua 2018**: mọi lựa chọn vẫn chỉ dựa trên TRAIN + VALIDATION 2012–2017.

10. Phân tích lỗi chi tiết

Nguồn: `reports/figures/evaluation_results.json` → `error_analysis`. Mọi phân khúc đều kèm số mẫu `n`.

10.1 Theo giờ trong ngày

Giờ	n	MAE Ridge	MAE baseline	Lưu lượng thực TB	Dự báo TB	Bias
00:00	273	176,39	110,72	831	865	+34,30
03:00	270	142,14	31,14	372	408	+36,26
07:00	271	355,75	422,46	4.813	4.811	−2,46
08:00	272	**376,07**	434,41	4.630	4.610	−20,23
16:00	273	**408,80**	485,32	5.794	5.678	−115,36
17:00	273	373,38	422,64	5.329	5.350	+20,51
22:00	273	348,61	323,83	2.234	2.214	−19,94

(bảng rút gọn; đầy đủ 24 giờ nằm trong `evaluation_report.md` và trên màn Dashboard)

Đọc kết quả:

• MAE lớn nhất rơi vào **16:00 (408,80)** và **08:00 (376,07)** — cũng là hai lúc lưu lượng lớn nhất. Sai số **tương đối** ở đây nhỏ (≈ 7 %) nhưng **tuyệt đối** lớn.

• Ở giờ đêm, **baseline lại tốt hơn Ridge** ở 7/24 giờ (00:00–04:00, 22:00–23:00); rõ nhất là 03:00: baseline 31,14 so với Ridge 142,14. Đây là phát hiện trung thực: ở vùng lưu lượng thấp và ít biến động, bảng trung bình theo `giờ × thứ` đã gần như tối ưu, còn Ridge bị kéo bởi biến thời tiết. Kiểm chứng trên dev + hướng phát triển: **§10.7**.

• Ridge vượt baseline ở **17/24 giờ**, và vượt rõ ở vùng cao điểm — nơi giá trị thực tế tập trung.

!MAE theo giờ và ngày trong tuần trên FINAL TEST 2018

Hình 5 — MAE theo giờ × ngày trên FINAL TEST 2018. Nguồn: `src/evaluate.py`.

10.2 Theo ngày trong tuần

Thứ	n	MAE Ridge	MAE baseline	Lưu lượng thực TB	Bias
Thứ 2	936	**347,28**	390,36	3.261	+80,41
Thứ 3	933	226,68	239,49	3.542	+33,39
Thứ 4	935	210,36	224,95	3.630	+3,50
Thứ 5	932	230,60	234,97	3.730	−25,98
Thứ 6	935	221,27	236,16	3.772	−58,86
Thứ 7	927	321,27	322,40	2.880	−58,84
Chủ nhật	935	260,93	262,05	2.451	−44,49

Ridge vượt baseline ở **cả 7/7 ngày trong tuần** (dù chỉ nhỉnh ở Thứ 7 và Chủ nhật). Sai số lớn nhất ở **Thứ 2 (347,28)** — ngày làm việc đầu tuần có biến động cao nhất. Ở Thứ 6 lưu lượng thực cao nhất (3.772) nhưng MAE thấp (221,27), bias âm (−58,86).

10.3 Ngày lễ — nơi mô hình yếu rõ rệt

Phân khúc	n mẫu	MAE Ridge	MAE baseline	RMSE	Lưu lượng thực TB	Dự báo TB	Bias
Ngày thường	6.366	239,49	250,58	362,87	3.347	3.335	−11,86
Ngày lễ	**167**	**1.031,44**	1.123,43	1.332,59	2.453	2.512	+58,68

• Chênh lệch MAE: **+791,95** — gấp **4,3 lần** ở ngày thường.

• Ở ngày lễ, mô hình **dự báo cao hơn thực tế 58,68 xe/giờ** (thiên lệch dương): mô hình chưa học

được việc lưu lượng ngày lễ **tụt mạnh** so với ngày thường, nên dự báo như một ngày làm việc hơi nhẹ.

• Ridge vẫn tốt hơn baseline ở cả hai phân khúc — nhưng **cả hai mô hình đều yếu** ở ngày lễ.

[HẠN CHẾ] Trong FINAL TEST 2018 chỉ có **7 ngày lễ** (167 giờ). Ước lượng trên một mẫu nhỏ như vậy có độ bất định lớn; con số 1.031,44 không nên đọc như một giá trị ổn định. Bảng MAE theo từng ngày lễ cho thấy các ngày lễ khác nhau gần gấp 3 lần (518,33 → 1.508,73) — xem §10.9.

10.4 Thời tiết

Thời tiết	n mẫu	MAE Ridge	MAE baseline	Bias
Clear · Clouds	2.328 · 1.996	245,15 · 243,39	256,64 · 254,11	−23,27 · −57,30
Rain · Drizzle · Thunderstorm	992 · 278 · 268	246,88 · 207,16 · 266,76	254,91 · 211,26 · 269,18	+14,83 · +31,50 · +79,74
Mist · Haze	1.099 · 272	295,41 · 313,56	319,87 · 332,90	+79,46 · +3,60
Fog	192	**536,05**	640,17	**+315,75**
Snow	521	**524,50**	585,36	**+238,88**
Smoke	2	137,93	88,11	+137,93
Squall	**0**	—	—	—

Các phân khúc còn lại đều có mẫu đủ, trừ `Smoke` (2 mẫu — thận trọng) và `Squall` (0 mẫu — không đánh giá được).

(Bảng đầy đủ 11 danh mục nằm ở `evaluation_report.md` và trên màn Dashboard.)

Các phân khúc dùng **multi-hot** nên chúng có thể trùng nhau (một giờ vừa mưa vừa tuyết được tính vào cả hai), tổng `n` có thể vượt 6.533.

Hai phát hiện đáng chú ý: Sương mù (Fog) tệ nhất về bias (+315,75 xe/giờ) — có thể vì khi sương mù, cách lưu lượng thay đổi không được mô tả bởi 5 biến thời tiết mà mô hình đang có. Và nguyên tắc **không có dữ liệu ≠ dự báo bằng 0**: `Squall` có `n = 0` nên báo *"không có mẫu trong FINAL TEST — không đánh giá được"*, còn `Smoke` chỉ có 2 mẫu (baseline còn tốt hơn) nên artifact gắn nhãn *"mẫu nhỏ — thận trọng"* và số 137,93 không mang thông tin.

!MAE theo nhóm thời tiết trên FINAL TEST 2018

Hình 6 — MAE theo nhóm thời tiết trên FINAL TEST 2018. Nguồn: `src/evaluate.py`.

10.5 Thời tiết cực đoan

Phân khúc	n mẫu	MAE Ridge	MAE baseline	RMSE	Bias
Có tuyết	521	**524,50**	585,36	784,62	+238,88
Có giông	268	266,76	269,18	409,84	+79,74
Có bão tố (Squall)	0	—	—	—	—
Bất kỳ hiện tượng cực đoan	**784**	**434,08**	475,09	675,09	+180,94
Không có hiện tượng cực đoan	5.749	235,96	245,32	367,75	−36,10

Khi có tuyết, MAE **gấp 2,22 lần** so với không có hiện tượng cực đoan; bias **+238,88** nghĩa là mô hình **dự báo cao hơn thực tế gần 239 xe/giờ** trong những giờ có tuyết. Giải thích hợp lý: trong tuyết, nhiều xe không ra đường, nhưng mô hình chỉ có 1 giờ dữ liệu thời tiết để suy ra điều đó.

10.7 Mô hình kém ở giờ đêm — mô tả, bằng chứng và hướng phát triển

Phần 2018 sinh bởi `src/uncertainty_audit.py` (§3 của `uncertainty_audit.md`); phần kiểm chứng trên dev sinh bởi `src/experiments.py` (mục 4 và 5 của `experiments_report.md`).

Mô tả hiện tượng

Trên FINAL TEST 2018, Ridge kém hơn baseline ở **7/24 giờ**, tất cả đều là giờ thấp điểm hoặc khuya:

Giờ	n	Lưu lượng thực TB	MAE Ridge	MAE baseline	MAE tương đối R / B
00:00 · 01:00	273 · 273	830,62 · 505,97	176,39 · 160,99	110,72 · 69,01	0,2124 · 0,3182 / 0,1333 · 0,1364
02:00 · 03:00	264 · 270	380,31 · 371,57	**157,06 · 142,14**	**52,95 · 31,14**	0,4130 · 0,3825 / 0,1392 · 0,0838
04:00 · 22:00 · 23:00	272 · 273 · 273	735,43 · 2.233,71 · 1.512,34	120,63 · 348,61 · 308,77	77,22 · 323,83 · 256,17	0,1640 · 0,1561 · 0,2042 / 0,1050 · 0,1450 · 0,1694

MAE tương đối (MAE chia cho lưu lượng thực trung bình của chính giờ đó) cho thấy đây **không chỉ** là hiệu ứng quy mô: ở giờ 00–04, sai số tương đối của Ridge là **0,2674** so với **0,1208** của baseline — gần gấp đôi; trong khi ban ngày

hai bên gần nhau (Ridge 0,0712, baseline 0,0807).

Bằng chứng: hiện tượng này CÓ THẬT, không chỉ riêng năm 2018

Đây là bước kiểm chứng quan trọng: một mẫu duy nhất (2018) không đủ để nói đó là đặc tính của mô hình. Vì vậy nhóm đo lại **cùng một phép so sánh** trên các cửa sổ dev out-of-sample:

Cửa sổ dev	n	Ridge kém ở mấy giờ	Trong đó giờ đêm	MAE đêm R / B	MAE tương đối đêm R / B	MAE tương đối ban ngày R / B
fold 1 — test 2015	3.593	18/24	**5/5**	210,54 / 73,27	0,3679 / 0,1280	0,0896 / 0,0852
fold 2 — test 2016	7.838	20/24	**5/5**	182,30 / 103,07	0,3102 / 0,1754	0,0993 / 0,0962
fold 3 — test 2017 (= VALIDATION 2017)	8.713	7/24	**5/5**	160,81 / 82,24	0,2681 / 0,1371	0,0734 / 0,0804

[QUYẾT ĐỊNH] Kết luận: Ridge kém ở **toàn bộ 5 giờ đêm trong cả 3/3 cửa sổ dev**. Vậy đây là **đặc tính của mô hình**, không phải đặc điểm ngẫu nhiên của năm 2018. Mô hình chỉ hơn baseline ở những giờ có lưu lượng lớn và biến động mạnh.

Giả thuyết

> Mô hình **cộng tuyến tính** nên hiệu ứng tháng và thời tiết được cộng thêm một lượng **tuyệt đối** gần như không đổi ở mọi giờ. Thực tế một cơn mưa giảm lưu lượng **tương đối** (ví dụ 20 %): 20 % của 3.500 xe/giờ là 700 xe, còn 20 % của 400 xe/giờ chỉ là 80 xe. Ở giờ đêm — nơi lưu lượng thấp — cùng hệ số tuyệt đối ấy lại **quá lớn**, kéo dự báo lệch nhiều hơn. Baseline không có hiệu ứng thời tiết nào để bị kéo theo, nên ở vùng lưu lượng thấp và ít biến động, bảng trung bình gần như đã tối ưu.

□□ **Đây vẫn là giả thuyết.** Nhóm đã thử kiểm chứng nó bằng log-target.

Kiểm chứng giả thuyết: log-target (khám phá hậu nghiệm, KHÔNG thay mô hình chính)

Nếu hiệu ứng thật sự là *tương đối*, biến đổi `log1p(traffic_volume)` làm nó thành tương đối, và MAE giờ đêm phải giảm mạnh. Nhóm thử trên **chỉ dữ liệu dev**:

Cửa sổ dev · biến đích	MAE	MAE giờ đêm
VALIDATION 2017 · tuyến tính (mô hình chính)	**272,12**	160,81
VALIDATION 2017 · log1p	295,00	**85,84**
pseudo-test 2016–2017 · tuyến tính (mô hình chính)	**306,48**	176,97
pseudo-test 2016–2017 · log1p	320,09	**93,52**

(RMSE và R² đi cùng chiều MAE — xem mục 5 của `experiments_report.md`.)

Đọc kết quả — giả thuyết chỉ được ủng hộ một nửa: MAE giờ đêm **giảm gần một nửa** (160,81 → 85,84 và 176,97 → 93,52) — đúng như giả thuyết dự đoán — nhưng MAE **tổng thể lại xấu hơn** (272,12 → 295,00 và 306,48 → 320,09), vì `log1p` nén thang lưu lượng cao và làm mô hình sai ở vùng cao điểm, nơi chiếm phần lớn sai số tuyệt đối.

[QUYẾT ĐỊNH] Vì sao nhóm KHÔNG đổi mô hình chính: (1) giả thuyết này được nêu ra **sau khi đã nhìn** FINAL TEST 2018, nên chọn log-target bây giờ là **test-informed model selection** — đúng thứ mà toàn bộ phương pháp đồ án này cảnh báo; (2) kết quả còn phụ thuộc mô hình và cần một quyết định dài hạn về ưu tiên vận hành mà nhóm **không** có cơ sở để đưa ra từ dữ liệu hiện có; (3) muốn dùng thì phải đánh giá lại trên **một holdout mới**, không phải trên 2018 đã xem.

Hướng phát triển (chưa thực hiện)

Hướng	Vì sao hợp lý	Cảnh báo
Tương tác `giờ × thời tiết`	Cho phép hệ số thời tiết khác nhau theo từng khung giờ, thay vì một hệ số chung — sát với giả thuyết hơn log-target và không phá thang lưu lượng cao	Phải chọn trên validation; cần cẩn thận với mẫu nhỏ ở giờ đêm
Mô hình phi tuyến (`HistGradientBoosting`)	Tự học tương tác mà không cần đặc trưng thủ công; ở 18/24 giờ Ridge vẫn hơn baseline nên có thể giữ được lợi thế ban ngày	Dễ rơi vào bẫy chọn mô hình theo test; đề tài thiên về minh hoạ phương pháp
Log-target có chọn lọc (ví dụ chỉ dùng cho giờ thấp)	Kết quả ở trên cho thấy log-target thắng rõ ở giờ đêm và thua ở giờ cao điểm	Cách chia nhánh phải chốt từ validation, không từ 2018

10.9 Giới hạn của tập FINAL TEST — thiếu quý IV và mẫu ngày lễ quá nhỏ

Phần định lượng trên VALIDATION sinh bởi `src/experiments.py` (mục 7); phần mô tả trên 2018 sinh bởi `src/uncertainty_audit.py` (mục 4).

(a) FINAL TEST không có tháng 10–12 — và điều đó làm số liệu *thuận lợi*

FINAL TEST 2018 kết thúc **30/09**, trong khi ba tháng tệ nhất của năm lại rơi vào cuối năm (MAE validation 2017: Dec 395,14 · Nov 344,52 · Jan 301,71).

Định lượng hệ quả — dùng chính VALIDATION 2017 làm phép thử, cùng một mô hình, chỉ khác khoảng thời gian chấm:

Khoảng chấm trên VALIDATION 2017	n	MAE
Cả năm 2017	8.713	272,12
Chỉ Jan–Sep 2017 (cùng phạm vi với FINAL TEST 2018)	6.513	**250,96**
Chỉ Oct–Dec 2017	2.200	334,78

Chênh lệch: −21,16 xe/giờ — MAE của Jan–Sep **thấp hơn** cả năm gần 8 %.

[QUYẾT ĐỊNH] Cách đọc đúng: số 259,73 của FINAL TEST 2018 có xu hướng **thấp hơn** so với một bài toán cả năm. Điều này **không** làm sai lệch phép so sánh với baseline (cả hai cùng bị thiếu quý IV, nên chênh lệch 13,16 vẫn công bằng), nhưng **không được** so sánh tuyệt đối con số này với một benchmark đánh giá cả năm — mọi kết luận về 2018 chỉ áp dụng cho **9 tháng đầu năm**.

(b) Ngày lễ trong FINAL TEST chỉ có 7 ngày lịch (167 giờ)

Ngày	Tên lễ	Số giờ	MAE Ridge	MAE baseline
2018-01-01	New Years Day	24	1.346,79	1.816,00
2018-01-15	Martin Luther King Jr Day	24	679,28	656,76
2018-02-19	Washingtons Birthday	24	518,33	877,77
2018-05-28	Memorial Day	24	1.208,26	1.309,72
2018-07-04	Independence Day	24	1.508,73	1.828,42
2018-08-23	State Fair	23	985,11	258,48
2018-09-03	Labor Day	24	971,69	1.080,85

□□ **Cảnh báo khi diễn giải:** con số "MAE ngày lễ = 1.031,44" ở §10.3 là **trung bình của 7 ngày lịch**, mỗi ngày chỉ khoảng 24 mẫu giờ. Bảng trên cho thấy các ngày lễ **khác nhau rất xa** — từ 518,33 (Washingtons Birthday) tới 1.508,73 (Independence Day), gần gấp 3 lần — nên **độ bất định của ước lượng này lớn**, không nên coi 1.031,44 là một giá trị ổn định. Nguyên nhân: mô hình chỉ có một cờ nhị phân `is_holiday`, nên nó học được "một mức dịch chuyển trung bình" cho tất cả ngày lễ, trong khi mỗi ngày lễ một kiểu (State Fair khác hẳn ngày lễ liên bang) — ở State Fair (2018-08-23) baseline tốt hơn Ridge rất nhiều (258,48 so với 985,11), đủ để thấy mô hình xử lý ngày lễ **không đồng đều**.

(c) Phân khúc thời tiết không có mẫu

`Squall` có 0 mẫu trong FINAL TEST — báo cáo ghi *"không có mẫu — không đánh giá được"* chứ **không** báo số 0, và cơ chế này được kiểm tra bằng `test_error_segments_include_sample_sizes` (giống phân khúc Smoke 2 mẫu ở §10.4).

10.10 Kết luận phân tích lỗi

Mô hình **vượt baseline trên toàn bộ 7/7 ngày trong tuần và 17/24 giờ**, nhưng **sai số tập trung ở ba nơi, đều có lý do giải thích được**:

Nơi	MAE	Cơ chế
Giờ cao điểm 16:00 và 08:00	409 / 376	Lưu lượng lớn → sai số tuyệt đối lớn dù sai số tương đối nhỏ
Ngày lễ	1.031	Hành vi ngày nghỉ lệch mạnh so với ngày thường, mẫu chỉ 7 ngày
Có tuyết / có sương mù	525 / 536	Dữ liệu thời tiết chỉ 1 giờ, không đủ mô tả việc nhiều xe không ra đường

Cả ba đều là **hạn chế thật của mô hình**, được báo cáo trung thực thay vì giấu bằng cách chỉ trích chỉ số tổng. Riêng điểm giờ đêm đã được tách riêng và kiểm chứng trên 3/3 cửa sổ dev ở **§10.7**.

11. Thí nghiệm phát triển (2012–2017)

> **Cảnh báo phân loại.** Toàn bộ mục này chạy trong cửa sổ **2012–2017** (34.042 dòng), **không chứa dòng nào của năm 2018**. Đây là số liệu **phát triển phương pháp**, không phải kết quả trên FINAL TEST, và **không được trộn vào bảng kết luận chính thức ở §9**.

Bảng đối chiếu: 4 thí nghiệm BẮT BUỘC của đề ↔ mục báo cáo ↔ artifact / hình

#	Thí nghiệm bắt buộc của đề	Mục trong báo cáo	Khoá JSON / file artifact	Hình
1	**Random split và time split**	§11.1 (Thí nghiệm 1), §11.2 (1b), §11.3 (1c)	`reports/figures/experiments_results.json` → `experiment_1_random_vs_time_split`, `experiment_1b_leakage_controlled`, `experiment_1c_block_neighbour`	`eda_traffic_by_month_year.png`
2	**Baseline và mô hình**	§7.1 (baseline), §7.2 (Ridge), §9.1 (kết quả FINAL TEST)	`reports/figures/evaluation_results.json` → `final_test.baseline`, `final_test.ridge`; `models/model_metadata.json`	`final_mae_by_hour_dow.png`
3	**Drift theo năm / mùa (rolling-origin, PSI)**	§11.4 (Thí nghiệm 3 — rolling-origin), §11.7 (Thí nghiệm 3b — PSI)	`reports/figures/experiments_results.json` → `experiment_3_rolling_origin`, `experiment_3b_covariate_drift`	— (bảng số; hình `eda_weather_distribution.png` minh hoạ phân bố)
4	**Phân tích lỗi ngày lễ và thời tiết cực đoan**	§10.3 (ngày lễ), §10.4 (thời tiết), §10.5 (thời tiết cực đoan)	`reports/figures/evaluation_results.json` → `error_analysis.holiday`, `error_analysis.weather_main`, `error_analysis.extreme_weather`	`final_mae_by_weather.png`

□□ **Đừng nhầm "Thí nghiệm 4" giữa hai tài liệu.** "Thí nghiệm 4" trong `reports/figures/experiments_report.md` là **kiểm tra giả thuyết giờ đêm** (tương ứng §10.7 của báo cáo này) — **KHÁC** với **thí nghiệm 4 bắt buộc của đề** ở bảng trên (phân tích lỗi ngày lễ + thời tiết cực đoan). Số hiệu "Thí nghiệm 1–8" trong `experiments_report.md` là đánh số **nội bộ** của giai đoạn phát triển 2012–2017, không trùng với cách đánh số 4 thí nghiệm bắt buộc của đề.

11.1 Thí nghiệm 1 — random split so với time split

Câu hỏi nghiên cứu của đề: **hai cách đánh giá này chênh nhau bao nhiêu?** Vì mỗi cách gọi `random_split` với một seed khác nhau sẽ ra một tập test khác nhau, nhóm lặp thí nghiệm trên **5 seed cố định** và báo cáo trung bình ± độ lệch chuẩn.

Time split (train ≤ 2015, test 2016-01-01 → 2017-12-31, n = 16.551): MAE **306,48**, RMSE 472,61, R² 0,9422.

Random split (mỗi seed một tập test ngẫu nhiên rải rác 2012–2017, n = 5.107):

seed	MAE	RMSE	R²
11	296,75	479,64	0,9415
23	292,09	475,82	0,9430
37	291,58	476,43	0,9420
53	284,94	456,88	0,9476
71	293,11	469,48	0,9432

(Phân bố n dòng theo từng tháng của từng tập test nằm ở `experiments_report.md` mục 1.)

Hai phép đo, hai ý nghĩa khác nhau:

Phép đo	MAE (random − time)	min	max
(a) Mỗi arm dùng tập test riêng	**−14,79 ± 3,83**	−21,54	−9,73
(b) **Cùng một tập dòng đánh giá**	**−5,43 ± 0,81**	−6,72	−4,65

• **(a)** là cách so sánh "tự nhiên": mỗi arm dùng tập test của chính nó. Nhưng **hai tập test khác nhau về thành phần năm **(time split test = 2016–2017 còn nguyên; random split test = mẫu ngẫu nhiên rải rác 2012–2017), nên một phần chênh lệch đến từ việc đây là** hai bài toán khác nhau** — con số này không chứng minh được rò rỉ.

• **(b)** là phép so sánh **công bằng về cách chọn tập huấn luyện**: hai mô hình (một train theo thời gian, một train ngẫu nhiên) được chấm trên **đúng cùng một tập dòng** — tập test của random split.

[QUYẾT ĐỊNH] Cách trả lời câu hỏi nghiên cứu: trên cùng một tập dòng đánh giá, random split cho MAE thấp hơn **5,43 ± 0,81** điểm — tức đánh giá bằng random split **lạc quan quá mức** một cách đáng kể, và đáng kể ấy **vì** con số này chỉ bằng khoảng 41 % cải thiện 13,17 điểm mà mô hình đạt được so với baseline.

Vì sao chênh lại nhỏ hơn nhiều so với bài toán chuỗi thời gian thường gặp? Mô hình ở đây chỉ dùng đặc trưng **lịch** (giờ, thứ, tháng) và **thời tiết quan sát tại đúng giờ đó**; nó **không dùng đặc trưng lag** của `traffic_volume`. Vì vậy nó không có cơ chế nào để *nhớ* giá trị của một dòng khác. Phần lạc quan còn lại gắn với việc random split **nhìn**

thấy được cả tháng 11–12 của năm 2017 trong khi tập train theo thời gian chỉ tới 2015 — tức lợi thế về **mức độ khớp mùa/năm**, chứ không phải do *nhìn thấy hàng xóm*.

11.2 Thí nghiệm 1b — tách kích thước tập huấn luyện khỏi khoảng cách thời gian

[QUYẾT ĐỊNH] Bốn arm dùng **CHUNG một tập test** (nửa còn lại của năm 2017, chia bằng seed cố định — 4.357 giờ ở seed đầu tiên). Vì có yếu tố ngẫu nhiên, nhóm lặp trên **5 seed cố định** và báo trung bình ± độ lệch chuẩn.

Arm	Tập train	n train (trung vị)	Dòng từ 2017	MAE (TB ± SD)
A	tới 2015-12-31 (xa test 2 năm)	17.491	0	270,30 ± 2,32
B	tới 2016-12-31 (xa test 1 năm)	25.329	0	269,74 ± 2,53
C	tới 2016 + nửa 2017 ngẫu nhiên	29.685	4.356	**263,19 ± 2,19**
D	**ngẫu nhiên từ C, đúng số dòng của B**	**25.329**	3.726	**262,98 ± 2,21**

Chênh lệch MAE	Trung bình ± SD	min	max
C − A (xa nhất → gần nhất)	−7,11 ± 0,44	−7,74	−6,65
C − B (thêm nửa 2017)	**−6,55 ± 0,37**	−6,92	−6,10
D − B (**cùng kích thước** với B)	**−6,76 ± 0,39**	−7,31	−6,19

Đây là kết quả quan trọng nhất của mục này, và nó không ủng hộ cách kết luận cũ:

> Thêm dữ liệu 2017 làm MAE giảm 6,55 điểm. Nhưng khi **ép cho tập huấn luyện có đúng bằng số dòng của arm B** (arm D — cùng kích thước, chỉ khác ở chỗ dữ liệu 2017 lấy ngẫu nhiên), hiệu ứng vẫn còn **6,76** điểm. Phần chênh giữa hai cái chỉ **+0,21** điểm — gần bằng không.

[QUYẾT ĐỊNH] Đọc đúng như sau — mô tả cái đo được, không khẳng định nhân quả:

• Cải thiện đo được **không** giải thích được bằng "nhiều dòng hơn": nó xuất hiện ngay khi tập huấn

luyện đã có dữ liệu của chính năm 2017, dù số dòng không đổi.

• **Không** được gọi phần còn lại là "mô hình nhìn thấy hàng xóm". Mô hình không dùng đặc trưng lag

nên **không thể nhớ** giá trị của dòng lân cận. Các yếu tố còn lẫn trong phần dư gồm mức lưu lượng riêng của năm 2017, và việc có dữ liệu ở đúng các tháng của tập test.

11.3 Thí nghiệm 1c — "hàng xóm" theo khối liên tục

Để tách yếu tố thứ hai, nhóm làm thí nghiệm với **khối liên tục** thay vì chia ngẫu nhiên: các **tháng chẵn** của 2017 vào tập huấn luyện, các **tháng lẻ** làm tập test chung (4.398 giờ).

Arm	n train	Dòng từ 2017	MAE	RMSE	R^2
P1 — tới 2016-12-31	25.329	0	281,06	443,79	0,9500
P2 — tới 2016 + tháng chẵn 2017	29.644	4.315	**278,66**	442,36	0,9503

ΔMAE (P2 − P1) = **−2,40** — lợi ích của việc có dữ liệu 2017 mà dữ liệu đó nằm ở **các tháng khác** với tháng của dòng cần dự báo.

So sánh hai cách đưa 2017 vào tập huấn luyện:

Cách đưa 2017 vào train	ΔMAE
Rải rác toàn năm (arm C của Thí nghiệm 1b)	**−6,55 ± 0,37**
Chỉ lấy các tháng *khác* tháng của dòng cần dự báo (arm P2)	**−2,40**
Chênh lệch	**−4,15**

[QUYẾT ĐỊNH] Cách đọc: hiệu ứng đo được **không** phải do số dòng (Thí nghiệm 1b đã kiểm tra) và **không** phải do mức năm 2017 (cả hai cách đều có 2017 trong tập huấn luyện). Nó gắn với việc tập huấn luyện có dữ liệu ở **cùng tháng và gần giờ** với dòng cần dự báo.

□□ **Đây vẫn là mô tả, không phải bằng chứng nhân quả.** Arm C và P2 khác nhau cả về tháng được thay vào tập huấn luyện, và vì mô hình không dùng đặc trưng lag nên nó không thể "nhớ" giá trị dòng lân cận — cơ chế nào trong hai khả năng đều còn là **giả thuyết**.

Kết luận thay đổi so với cách diễn giải trước đây. Bản báo cáo cũ kết luận "MAE giảm 7,55 **chỉ vì** mô hình nhìn thấy hàng xóm". Thí nghiệm mới **không** ủng hộ phát biểu đó: con số cũ gộp lẫn ba yếu tố (kích thước tập huấn luyện, mức năm, khoảng cách thời gian); sau khi tách, phần lớn hiệu ứng **không** phải do kích thước; và cơ chế "nhìn thấy hàng xóm" **không khả thi với mô hình không có lag** — mô hình không nhìn thấy *giá trị* của dòng lân cận, chỉ thấy

đặc trưng lịch và thời tiết của nó.

Điều Thí nghiệm 1b/1c **vẫn** chứng minh được là: **khoảng cách thời gian giữa tập huấn luyện và tập dự báo làm thay đổi đáng kể con số đánh giá** (4,15 MAE chỉ từ việc dữ liệu 2017 nằm ở cùng tháng hay tháng khác). Đó mới là lý do trung thực để giữ FINAL TEST 2018 nguyên vẹn.

11.4 Thí nghiệm 3 — rolling origin (drift, chỉ out-of-sample)

Thiết kế: cửa sổ mở rộng (expanding window), mọi fold đều ngoài mẫu.

Fold	Năm test	Khoảng test	n	MAE	RMSE	R^2
1	2015	2015-06-11 → 2015-12-31	3.593	325,98	497,93	0,9362
2	2016	2016 cả năm	7.838	343,77	523,66	0,9274
3	2017	2017 cả năm	8.713	**272,12**	421,46	**0,9548**

Tổng hợp 20.144 giờ out-of-sample (2015–2017): **MAE = 309,61**.

MAE giảm 53,86 (16,5 %) từ fold 1 đến fold 3. Nhưng kết luận phải là:

> Chất lượng được cải thiện, **có thể** do năm gần nhất (2017) dễ hơn, **không nhất thiết** là mô hình tiến bộ. Ba điểm dữ liệu không đủ để phân biệt "mô hình tốt lên" với "năm 2017 dễ hơn".

[QUYẾT ĐỊNH] Tách bạch hai khái niệm mà nhiều báo cáo hay trộn: **performance theo tập** — mô hình tốt hơn không (so sánh ngoài mẫu trên cùng điều kiện) — và **drift** — dữ liệu có đổi không (kiểm tra phân bố, xem §11.7).

11.6 Thí nghiệm 8 — mở rộng lag (thí nghiệm độc lập, KHÔNG vào serving)

Sinh bởi `src/experiments.py` (mục 8). Chỉ dùng dữ liệu 2012–2017.

Cách dựng lag — và vì sao đây là toàn bộ vấn đề

Dữ liệu thiếu **22,79 %** số giờ, nên `df[target].shift(k)` theo **dòng** không hề là "lag k giờ" — đó là "k dòng trước", có thể cách nhau 1 giờ, 2 giờ, hay cả một tuần. Vì vậy nhóm dựng lag **đúng cách**: ghép theo **thời điểm** (`date_time − k giờ`) trên chuỗi đã sắp xếp, để NaN khi giờ đó không có quan sát, và **không** nội suy. Lag dùng: **1 giờ, 24 giờ, 168 giờ**; lọc bỏ **5.796 dòng (17,03 %)** vì thiếu giá trị lag — **không** loại dòng nào vì dữ liệu thiếu sẵn.

Kết quả — và kết quả này **không ủng hộ** giả thuyết của nhóm

Hai mô hình (train theo thời gian vs train ngẫu nhiên) được chấm trên **cùng một tập dòng đánh giá**, lặp 5 seed. Giá trị **âm** = random split trông tốt hơn = lạc quan.

Cách dựng lag	Độ lạc quan do random split (TB ± SD)	So với không lag
Không lag	**−3,48 ± 1,57**	—
Lag ĐÚNG (ghép theo thời điểm)	**−0,55 ± 0,35**	**+2,93**
Lag SAI (`shift()` theo dòng)	**−7,17 ± 0,81**	**−3,69**

Đọc đúng, theo đúng những gì đo được:

• □ **Lag dựng đúng KHÔNG làm tăng lạc quan** — nó *giảm* độ lạc quan (−3,48 → −0,55). Giả thuyết ban đầu của nhóm ("thêm lag thì random split sẽ lạc quan hơn nữa") **sai**: lag-1 tại thời điểm dự báo là một **quan sát quá khứ thật**, sẵn có ở cả hai cách chia, nên nếu tính đúng và **trước** khi tách tập thì nó không phải thông tin tương lai.

• □ **Nhưng kết quả này làm nổi bật đúng rủi ro thật:** chỉ cần dựng lag **sai** (`shift()` theo dòng) thì độ lạc quan **tăng gần gấp đôi** (−3,48 → −7,17) — với random split, dòng ngay trước dòng test nằm trong tập huấn luyện, nên `traffic_volume` của nó vừa là **nhãn huấn luyện** vừa là **đặc trưng** của dòng test.

> **Kết luận phương pháp quan trọng:** thứ cần kiểm soát là **cách tính lag**, không phải bản thân việc dùng lag. Đề tài vẫn giữ đúng quan điểm "lag là đường nghiệm dễ rơi vào rò rỉ thời gian" — nhưng bằng lý do **đo được**, không phải bằng phỏng đoán.

Lag có giúp không? Có — rất nhiều (và nhóm vẫn không dùng)

Trên **time split**, MAE giảm từ **298,74** (không lag) xuống **172,66** (lag đúng theo thời điểm) — giảm khoảng **42 %**. Đây là cải thiện rất lớn và có thật trên dữ liệu dev.

[QUYẾT ĐỊNH] Vì sao nhóm KHÔNG đưa lag vào mô hình chính: (1) FINAL TEST 2018 **đã được xem**, nên biết lag "có vẻ giúp nhiều" rồi thêm lag vào mô hình là **test-informed model selection** — đúng thứ toàn bộ phương pháp của đồ án này cảnh báo; (2) thêm lag còn kéo theo bài toán **train-serving skew**: tầng phục vụ phải nhận lưu lượng của `k` giờ trước, tức không còn là dự báo "chỉ từ lịch và thời tiết" như đề tài định nghĩa; (3) 5.796 dòng bị loại (17,03 %) khi thiếu lag — cần xử lý ở tầng phục vụ.

Hướng đúng cho công sau: đánh giá lại toàn bộ trên **một holdout mới**, có kiểm soát rõ cách dựng lag và kiểm thử train-serving skew cho đặc trưng lag.

11.7 Thí nghiệm 3b — kiểm tra dịch chuyển phân bố (PSI)

PSI so với năm tham chiếu 2013 (ổn định < 0,1; trung bình 0,1–0,25; mạnh > 0,25):

Biến	2012	2013	2014	2015	2016	2017
`traffic_volume`	0,0210	0,0000	0,0041	0,0298	0,0345	0,0196
`temp`	**1,1035**	0,0000	0,0794	**0,7389**	0,1619	0,1027
`clouds_all`	0,3329	0,0000	0,0593	**0,6980**	0,0715	0,3532

Đọc kết quả:

• Phân bố **mục tiêu** rất ổn định (PSI < 0,035 mọi năm) → hành vi giao thông không dịch chuyển.

• Phân bố **nhiệt độ và mây** dịch chuyểch mạnh ở 2012 và 2015 (PSI 0,70–1,10) → có thể do mùa và

khí hậu, và do **tập 2012 chỉ có 3 tháng** (từ tháng 10) còn 2015 thiếu quý I–II. Đây là hạn chế của phép so sánh PSI khi tập không cùng độ dài.

• PSI của biến đầu vào cao **không** tự động nghĩa mô hình hỏng; nó là tín hiệu cần theo dõi,

và ở đây kết hợp với §11.4 cho thấy hiệu năng vẫn ổn định.

!Phân bố thời tiết trên TRAIN

Hình 7 — Phân bố hiện tượng thời tiết, TRAIN. Nguồn: `src/eda.py`. Đây là cơ sở để chọn mã hoá multi-hot thay vì một nhãn đơn.

12. Ứng dụng Web / API

12.1 Nguyên tắc thiết kế — không train-serving skew

Đây là yêu cầu khó nhất khi đưa mô hình ra phục vụ. Năm rủi ro và cách xử lý:

Rủi ro	Cách xử lý của nhóm	Bằng chứng
Backend tạo feature khác lúc train	Dùng **CHUNG** hàm `src.features.build_features`	`test_no_train_serving_skew_on_real_rows` so từng cột trên dữ liệu thật
Backend lấy `is_holiday` bằng cách quét dataset	Dùng **lịch tất định** `src/holidays.py`	`test_holiday_flag_applies_to_whole_day_not_only_midnight`
Backend fit lại encoder/imputer/scaler	**Không bao giờ** — chỉ `predict` trên pipeline đã đóng băng	`test_pipeline_is_not_refit_at_serving`
Backend đọc target từ dataset	**Không** — API chỉ cần `models/`	`test_server_reads_no_target_at_serving` (trỏ `data/` vào thư mục rỗng, API vẫn chạy)
Backend tự chọn lại alpha	**Không** — alpha cứng trong artifact	`model-info.serving.tunes_at_serving = false`

[QUYẾT ĐỊNH] Ứng dụng **tải artifact lúc khởi động và cache lại**, không nạp lại mỗi request. `src/train.py` là nơi duy nhất được phép ghi vào `models/`.

12.2 Schema thân thiện

Người dùng chỉ cần nhập **thông tin thực sư có ở thời điểm dư báo**:

```
{
"date_time": "2018-06-15T08:00:00",
"temperature_celsius": 20.0,
"rain_1h_mm": 0.0,
"snow_1h_mm": 0.0,
"clouds_all": 20,
"weather": ["Clear"]
}
```

Backend tự suy ra toàn bộ đặc trưng kỹ thuật hoá: giờ, thứ, tháng, `is_weekend`, `hour_dow`, ****is_holiday` **(từ lịch tất định — người dùng** không** nhập tay), `weather_main_mode`, `weather_family`, `weather_severity` và 11 cột multi-hot. `rain_1h_mm` / `snow_1h_mm` **tuỳ chọn** (mặc định 0); `weather` nhận **nhiều hiện tượng cùng lúc**; nhiệt độ nhận **°C** và backend cộng 273,15 ra Kelvin; `state_fair_start_date` **tuỳ chọn**, chỉ dùng khi dự báo nằm ngoài bảng lịch.

12.3 Ba màn hình

Màn	Đường dẫn	Nội dung
1	`/`	Tên đề tài, dữ liệu, vị trí I-94 westbound / ATR 301, mục tiêu, đơn vị xe/giờ, phạm vi dùng, giới hạn
2	`/du-bao`	Form → gọi **thật** `POST /api/traffic-forecast`; có trạng thái loading / thành công / lỗi validation / lỗi server
3	`/dashboard`	Toàn bộ số liệu đánh giá, phân tích lỗi, thí nghiệm, Model Card

Giao diện tiếng Việt, không thư viện ngoài, responsive (đã kiểm tra ở 1440 px và 820 px).

Không có số liệu nào được hard-code trong HTML/JS. Màn 3 lấy mọi con số từ `GET /api/dashboard-metrics`, màn 2 lấy danh mục thời tiết hợp lệ từ `GET /api/model-info`. Test `test_no_final_test_metric_is_hardcoded_in_frontend` quét từng file frontend và đối chiếu với artifact để bảo đảm điều này không thoát.

12.4 Validation

Trường	Quy tắc	Lý do chọn mức
`date_time`	ISO-8601, năm 1900–2200	chỉ bắt lỗi định dạng
`temperature_celsius`	**−70 … 70 °C**	ngoài khoảng này gần như chắc chắn là nhập nhầm Kelvin hoặc °F
`rain_1h_mm`, `snow_1h_mm`	**≥ 0**, ≤ 500	lượng 1 giờ không âm; trần 500 bắt lỗi nhập tổng cả ngày
`clouds_all`	số nguyên **0 … 100**	đúng định nghĩa %
`weather`	danh sách **không rỗng**, chỉ 11 danh mục	sai chính tả (vd `clear`) cũng bị từ chối, **không tự sửa**
trường lạ	bị từ chối	`extra="forbid"` — không bỏ qua âm thầm

Mọi lỗi trả **HTTP 422** với body:

```
{
"error": "validation_error",
"message": "Input không hợp lệ: Input should be less than or equal to 100",
"details": [{ "field": "clouds_all", "message": "...", "type": "less_than_equal" }]
}
```

Server không crash: có exception handler ở mọi tầng; test `test_server_survives_a_burst_of_bad_requests` gửi liên tiếp 5 payload sai rồi xác nhận request hợp lệ vẫn trả 200.

12.5 Chính sách dự báo của tầng phục vụ (đã đóng băng)

Ridge là hồi quy tuyến tính nên **có thể** trả giá trị âm. Lưu lượng xe không thể âm — nhưng đây là quyết định dễ sinh **test-informed postprocessing**: nếu nhìn FINAL TEST 2018 thấy "cắt đi, MAE đẹp hơn" rồi mới quyết định cắt, thì kết luận 2018 mất ý nghĩa. Nhóm **tách quyết định khỏi đo lường** và đóng băng quyết định trước.

12.5.1 Hai thứ phải phân biệt, và quy trình quyết định trước – đo sau

	Là gì	Ý nghĩa
RAW MODEL	Ridge(`alpha` = 0,001, solver `lsqr`) trả về trực tiếp	**Metric của mô hình.** Đây là kết luận chính thức, xuất hiện ở §9 và §14.5.
DEPLOYED PREDICTOR	`max(0, ·)` □ Ridge	Giá trị mà API trả về. Là **wrapper phục vụ**, KHÔNG phải mô hình khác.

Hai hàng số này **không phải cùng một model metric** và không được gọi chung tên. Mọi con số trong báo cáo nêu là "kết quả của mô hình" đều là số của **RAW MODEL**.

Thứ tự chạy **bắt buộc** của nhóm là: `train` → `experiments` → **`freeze_serving_policy` → `evaluate`** → `postprocess_audit`. Nói riêng về chính sách phục vụ:

Bước	Script	Dữ liệu dùng	Việc làm
1	`src/freeze_serving_policy.py`	**TRAIN + VALIDATION**	**Quyết định** → ghi `models/serving_policy.json`
2	`src/evaluate.py`	FINAL TEST 2018	Mở 2018 ra đánh giá — **sau khi** policy đã đóng băng
3	`src/postprocess_audit.py`	FINAL TEST, chạy **sau** bước 2	**Chỉ đo lại** hậu quả để báo cáo minh bạch

Nếu đảo bước 1 và 2 thì chính sách hậu xử lý thành **test-informed postprocessing**: ta chọn cách hậu xử lý *vì* đã nhìn thấy kết quả 2018, và FINAL TEST không còn là tập đánh giá độc lập nữa. Chính vì vậy `postprocess_audit.py`

và `uncertainty_audit.py` **từ chối chạy** nếu chưa có policy đã đóng băng, hoặc nếu artifact tự ghi `final_test_used_for_selection` khác `False`. Có test `test_freeze_script_source_never_reads_final_test` kiểm tra ở mức mã nguồn rằng `build_policy()` chỉ gọi bằng chứng cho `train` và `validation`, và `test_pipeline_order_freezes_policy_before_final_test` kiểm tra mọi tài liệu bàn giao đều mô tả đúng thứ tự này.

12.5.2b Đóng băng phải chạy lại được (idempotent)

"Đóng băng" chỉ có ý nghĩa nếu quyết định được ghi **một lần**. Ban đầu script ghi `frozen_at_utc = datetime.now(...)` ở mỗi lần chạy, nên `serving_policy.json` **đổi hash dù quyết định không đổi**. Nay `build_policy()` trả về **nội dung** quyết định (trừ `frozen_at_utc`), nên kết quả là hằng số: khớp thì **không ghi lại file**; lệch thì in diff và dừng (exit ≠ 0) trừ khi có `--force`. `--check` chỉ tính lại rồi so sánh, **không ghi file nào**. Chi tiết ở README.

12.5.3 Quy tắc quyết định (viết trước khi chạy)

Adopt `max(0, ·)` **khi và chỉ khi** cả ba điều kiện sau đều đúng, tất cả đo trên TRAIN + VALIDATION:

Mã	Điều kiện	Kết quả
D1	min(`traffic_volume`) trên **TRAIN** >= 0	**PASS** (sàn = 0,00)
D2	Mô hình có thực sự trả dự báo thô < 0	**PASS** (TRAIN 187 dòng, VALIDATION 58 dòng)
D3	MAE sau khi cắt <= MAE thô trên **VALIDATION 2017**	**PASS** (272,12 → 269,55)

Nếu một điều kiện FAIL, script ghi `policy = "none"` — **không cắt**, và negative prediction được ghi thẳng là limitation (đã viết sẵn nhánh này: `test_freeze_script_has_fallback_to_no_policy`).

12.5.4 Căn cứ đo được — miền giá trị và VALIDATION 2017

`traffic_volume` là số xe đi qua một trạm đo trong một giờ, nên **không thể âm**; min trên **TRAIN** = **0,00** □ sàn miền giá trị đúng là 0 (min trên VALIDATION = 186,0; min trên FINAL TEST = 151,0 — **không** dùng để quyết định).

Chỉ số trên VALIDATION 2017	RAW MODEL	DEPLOYED PREDICTOR (`max(0,·)`)
Số dự báo thô âm	**58** / 8.713 (0,67 %)	—
Dự báo thô nhỏ nhất	**−790,56**	—
MAE	272,12	**269,55**
RMSE	421,46	416,73
R^2	0,9548	0,9558

Lưu lượng thực trung bình trên các dòng có dự báo âm: **529,91** — sự thật vẫn **dương**, nên cắt về 0 là sửa sai lệch của mô hình, không phải xoá sự thật.

12.5.6 Căn cứ bổ sung — lập luận toán học (không cần dữ liệu)

> Với sự thật $y >= 0$ và sàn $b = 0$: $|y - max(0, \hat{y})| <= |y - \hat{y}|$ **với mọi giá trị ŷ**. Chứng minh: nếu $\hat{y} >= 0$ thì hai vế bằng nhau; nếu $\hat{y} < 0$ thì $|y - 0| = y$ và $|y - \hat{y}| = y - \hat{y} > y$.

Kiểm chứng thực nghiệm trên VALIDATION 2017: sai số tuyệt đối **không tăng ở bất kỳ dòng nào** (`pointwise_abs_error_never_increases = True`), và **đúng 58 dòng** bị thay đổi — bằng đúng số dòng có dự báo âm. Đây là **phép chiếu đúng theo miền giá trị**, không phải siêu tham số được học từ dữ liệu, nên không vi phạm nguyên tắc "không học gì từ tập test".

12.5.7 Báo cáo tác động trên FINAL TEST (chạy SAU khi đã đóng băng)

Tập	n dự báo thô âm	min thô	MAE — RAW MODEL	MAE — DEPLOYED	RMSE — RAW	RMSE — DEPLOYED
VALIDATION 2017	58 (0,67 %)	−790,56	272,12	269,55	421,46	416,73
FINAL TEST 2018	**34 (0,52 %)**	**−911,35**	**259,73**	**257,54**	**416,78**	**412,25**

Cả 34 dòng ở FINAL TEST đều rơi vào **giờ 0–4 ban đêm**, **32/34 là ngày lễ**, lưu lượng thực trung bình 572,56 xe/giờ. Policy vì thế **không phải hình thức**, đồng thời chỉ ra điểm yếu đã nêu ở §10.3: mô hình chưa học được việc lưu lượng đêm ngày lễ gần bằng 0.

12.5.8 Phát biểu trung thực về số liệu

> Số ở §9 (MAE 259,73 · RMSE 416,78 · R^2 0,9554) là **RAW MODEL** — kết quả của gói đánh giá đã đóng băng `src/evaluate.py`. Số 257,54 là **DEPLOYED PREDICTOR** — giá trị API trả về sau khi chiếu về sàn; nó là số của một *wrapper*, không phải của mô hình, và **không được trích dẫn như "hiệu năng mô hình"**. Nhóm **không** chọn policy vì nhìn thấy 34 dự báo âm trên FINAL TEST: quyết định được chốt trên TRAIN + VALIDATION theo quy tắc D1–D3 (§12.5.3), và nhóm **không** sửa lại gói đánh giá đã đóng băng để khớp API — làm vậy tức là dùng kết quả

2018 để điều chỉnh mô hình.

12.6 Thiếu artifact

Nếu `models/ridge_pipeline.joblib` hoặc `model_metadata.json` không có:

• `/health` → **HTTP 503**, `status = "degraded"`, kèm tên file thiếu và lệnh cần chạy;

• `/api/traffic-forecast`, `/api/model-info` → **HTTP 503** với

`error = "model_artifact_unavailable"`;

• server **không** sập, và **không** tự huấn luyện (đó là nguyên tắc của dự án).

Test `client_without_artifacts` chỉ vào thư mục model rỗng để kiểm chứng.

13. Kiểm thử tự động

Kết quả: `py -m pytest tests\ -v` → 392 passed, 0 failed.

File	Số test	Phạm vi
`test_data.py`	39	collapse trùng, quy tắc giá trị vô lý, `keep_default_na=False`, đối chiếu lịch với dataset
`test_features.py`	19	Đặc trưng lịch, ngữ nghĩa ngày lễ, time split có assert, random split chỉ để minh hoạ
`test_pipeline.py`	20	`SimpleImputer` trong pipeline và trước scaler, mọi bộ tiền xử lý fit TRAIN only, xử lý NaN
`test_experiments.py`	36	Chặn FINAL TEST 2018 (`assert_no_final_test_rows`), arm chung tập test, tái lập theo seed, phân tích giờ đêm / log-target / alpha chỉ trên dev
`test_serving.py`	29	°C→K, `is_holiday` tự tính, multi-hot, **không train-serving skew**, chính sách hậu xử lý
`test_api.py`	85	Route, `/health`, dự báo hợp lệ, 12 ca validation sai, cảnh báo khi thiếu mưa/tuyết, web route 200, dashboard lấy số từ artifact
`test_serving_policy.py`	49	Policy không test-informed, điều kiện D1–D3, phân biệt RAW MODEL vs DEPLOYED PREDICTOR, **đóng băng lặp lại được** (`--check` / `--force`), dung sai float theo khối `evidence`
`test_report.py`	99	Tài liệu khớp artifact, không bịa số, số test đồng bộ, thứ tự pipeline, chính tả "rò rỉ"
`test_uncertainty_audit.py`	16	Bootstrap theo khối ngày tái lập được, script chỉ đo, kết luận khớp số liệu, cửa sổ dev không có 2018
Tổng	**392**	

Con số này không được gõ tay. `tests/test_report.py::test_documented_test_count_matches_real_collection` chạy `pytest --collect-only` trên chính bộ test rồi bắt README, báo cáo, slide, kịch bản demo và câu hỏi vấn đáp phải nói đúng số đó. Thêm hay bớt một test mà quên sửa tài liệu là **test đỏ**, không thể lọt.

13.1 Những test đáng chú ý nhất

Test	Điều nó chứng minh
`test_no_train_serving_skew_on_real_rows`	Dựng feature từ dòng thật theo đường dẫn serving, so **từng cột** với đường dẫn train. Không cột nào lệch.
`test_pipeline_is_not_refit_at_serving`	Gọi API 8 lần, kiểm tra `scaler.mean_` và `encoder.categories_` **không đổi**
`test_server_reads_no_target_at_serving`	Trỏ `data/` vào thư mục rỗng → API vẫn dự báo được □ không đọc dataset
`test_error_segments_include_sample_sizes`	Phân khúc `n = 0` phải có `MAE = null`, không được báo 0
`test_documented_test_count_matches_real_collection`	Chạy `pytest --collect-only` rồi bắt tài liệu phải nói đúng số test đó

13.2 Cấu hình cho người chạy

Ba biến môi trường cho phép trỏ ứng dụng sang thư mục khác (dùng trong test, không cần cho người dùng):

`TRAFFIC_MODELS_DIR`, `TRAFFIC_REPORTS_DIR`, `TRAFFIC_DATA_DIR`.

14. Model Card

14.1 Tóm tắt mô hình

Tên	`ridge_pipeline` (phiên bản `checkpoint3`)
Loại	Hồi quy Ridge (L2) — `sklearn.linear_model.Ridge`
Hyperparameter	`alpha = 0,001`, `solver = "lsqr"`
Tiêu chí chọn alpha	MAE trên VALIDATION 2017
Artifact	`models/ridge_pipeline.joblib`
Số đặc trưng sau tiền xử lý	217
Target	`traffic_volume` — lưu lượng, đơn vị **xe/giờ**
Tạo bởi	`src/train.py`, seed = 42

14.2 Dữ liệu

Nguồn	Metro Interstate Traffic Volume, UCI ML Repository (CC BY 4.0)
Vị trí	I-94 chiều westbound, trạm ATR 301
Số dòng thô	48.204
Số dòng dùng để mô hình hoá	40.575 (sau khi collapse 7.629 dòng trùng timestamp)
Khoảng thời gian	2012-10-02 09:00 → 2018-09-30 23:00
Đơn vị quan sát	1 giờ tại một trạm đo

14.3 Chia tập

Tập	Khoảng	n
Train (huấn luyện)	2012-10-02 09:00 → 2016-12-31 23:00	25.329
Validation (chọn alpha)	2017-01-01 00:00 → 2017-12-31 23:00	8.713
Final holdout (chỉ đánh giá)	2018-01-01 00:00 → 2018-09-30 23:00	6.533

14.4 Đặc trưng & tiền xử lý

Danh sách đầy đủ ở §7.2.1. Tóm tắt: **4 cột categorical** (`OneHotEncoder`), **5 cột numeric** (`SimpleImputer(median)` → `StandardScaler`), **12 cột binary** (`passthrough`) → **217 cột** sau tiền xử lý. Mọi thống kê học được đều **fit trên TRAIN duy nhất**. Mô hình **không** dùng đặc trưng lag (xem §11.6).

14.5 Số liệu đánh giá (tất cả đều ngoài mẫu)

Tập / mô hình	MAE	RMSE	R^2	n
Validation 2017 — Ridge	272,12	421,46	0,9548	8.713
Validation 2017 — Baseline	278,66	461,00	0,9459	8.713
Final test 2018 — Ridge	**259,73**	**416,78**	**0,9554**	6.533
Final test 2018 — Baseline	272,90	473,13	0,9426	6.533

Không đưa MAE trên tập train vào bảng này: train là **in-sample**, hai dòng trên là **out-of-sample** — trộn chúng là so sánh không cùng đối tượng. Bốn dòng trên là số của **RAW MODEL** — Ridge trả về trực tiếp; đó là metric của *mô hình* và là kết luận chính thức.

[HẠN CHẾ] Đọc 4 dòng này như thế nào cho đúng. Ở FINAL TEST 2018, Ridge nhỉnh hơn baseline 13,16 MAE và khoảng tin cậy 95 % không chứa 0 (§9.3). Nhưng trên dữ liệu 2012–2017 (pseudo-test 2016–2017 và 3 fold rolling-origin), Ridge chỉ thắng ở **1/4 cửa sổ**. Vì vậy Model Card này **không** tuyên bố "mô hình tốt hơn baseline nói chung" — chỉ tuyên bố về cửa sổ 2018 và về một tập huấn luyện có dữ liệu đầy đủ.

Tầng phục vụ áp dụng thêm **DEPLOYED PREDICTOR** = `max(0, ·)` □ Ridge (chính sách đã đóng băng, xem §12.5). Số của nó được báo ở §12.5.7 và **không được gọi chung tên** với số ở bảng này:

	MAE	RMSE	R^2
RAW MODEL (Ridge) — *metric của mô hình*	**259,73**	**416,78**	**0,9554**
DEPLOYED PREDICTOR (`max(0,·)` `Ridge`) — *wrapper phục vụ*	257,54	412,25	0,9564

14.6 Mục đích sử dụng dự kiến

• Ước lượng sơ bộ lưu lượng theo giờ tại trạm ATR 301 khi chưa có số đo thực tế.

• So sánh kịch bản lưu lượng giữa các ngày có điều kiện thời tiết khác nhau.

• Minh hoạ giá trị của chia tập theo thời gian và đánh giá trung thực trên bài toán chuỗi thời gian thực tế.

14.7 Ngoài phạm vi sử dụng

• Điều khiển giao thông, điều khiển tín hiệu, định tuyến phương tiện.

• **Mọi quyết định an toàn (safety-critical)**.

• Suy rộng sang trạm khác, hướng khác, hoặc dùng làm đại diện cho toàn thành phố.

• Dùng như số liệu thống kê chính thức của cơ quan quản lý giao thông.

14.8 Hạn chế

• **Một trạm duy nhất** (ATR 301, I-94 westbound). Kết quả **không đại diện cho toàn thành phố** và

không suy rộng được cho trạm hoặc hướng khác.

• **Ngày lễ: MAE = 1.031,44** với n = 167 giờ (7 ngày lễ), so với 239,49 ở ngày thường

(chênh 791,95). Mô hình dự báo **cao hơn thực tế 58,68 xe/giờ** ở ngày lễ. □□ Mẫu chỉ có **7 ngày lịch** và các ngày lễ khác nhau rất xa (MAE từ 518,33 tới 1.508,73) → độ bất định lớn, không đọc như một giá trị ổn định. Ở State Fair (2018-08-23) baseline còn tốt hơn Ridge (258,48 so với 985,11). Chi tiết ở §10.9.

• **Thời tiết cực đoan: MAE = 434,08** (n = 784) so với 235,96 khi không có (n = 5.749) — gấp

1,84 lần. Riêng tuyết: MAE 524,50 (gấp 2,22 lần), bias +238,88. Sương mù còn tệ hơn: MAE 536,05, bias +315,75.

• **Final test chỉ tới 30/09/2018** — không có dữ liệu tháng 10–12/2018, và đây lại là ba

tháng khó nhất (MAE validation 2017: Nov 344,52 · Dec 395,14). Chấm Jan–Sep 2017 cho MAE 250,96, tức thấp hơn cả năm **21,16** → số 2018 được đo trên phần dễ hơn của năm (§10.9).

• **Dữ liệu lịch sử 2012–2018.** Mô hình không cập nhật theo thay đổi hạ tầng, chính sách giao

thông, giá nhiên liệu hay hành vi người dùng sau thời điểm này.

• **State Fair có phạm vi năm hữu hạn.** Bảng lịch chỉ có năm **2012–2020** vì ngày khai mạc do

bang công bố, không có quy tắc lịch. Ngoài khoảng này API **báo rõ thay vì tự đoán**; người dùng có thể truyền `state_fair_start_date` để tính đúng.

• **Không dùng cho mục đích safety-critical.**

• **Mô hình tuyến tính** — không nắm hiệu ứng phi tuyến phức tạp, và có thể lệch đáng kể ở tình

huống chưa từng xuất hiện trong dữ liệu huấn luyện.

• **Đầu vào thời tiết là thời tiết QUAN SÁT tại đúng giờ đó**, không phải thời tiết dự báo. Khi

triển khai dự báo thật cho tương lai, phải dùng **thời tiết dự báo**, và **sai số dự báo thời tiết sẽ cộng thêm vào sai số của mô hình** này. Mô hình **chưa** được đánh giá với thời tiết dự báo, nên mọi số liệu MAE/RMSE ở trên là **cận dưới** của sai số trong điều kiện vận hành thật.

• Ở **giờ đêm** (00:00–04:00, 22:00–23:00) baseline lại tốt hơn Ridge (ví dụ 03:00: baseline

31,14 so với Ridge 142,14) → Ridge không phải lựa chọn tối ưu cho mọi khung giờ. Chi tiết ở §10.7.

• **Dự báo thô âm** ở 34/6.533 dòng của FINAL TEST (đều giờ 0–4 ban đêm, 32/34 là ngày lễ, thấp

nhất −911,35). API chặn về 0 theo policy `max(0, ·)` và báo cờ `clipped_to_zero`; số liệu ở §14.5 tính trên dự báo **thô** (MAE 259,73), sau khi chặn là 257,54.

• **Hiệu ưu thế so với baseline không nhất quán theo thời gian.** Trên 2018 Ridge thắng

13,16 MAE (CI 95 % [+3,80; +22,00]); trên 3/4 cửa sổ dev 2012–2017 Ridge **thua** baseline với khoảng tin cậy nằm hẳn về phía baseline (§9.3). Mô hình chỉ đáng tin hơn khi tập huấn luyện đủ dày.

• **Hiệu chuẩn L2 gần như không có tác dụng.** Trên validation, `alpha ∈ [0; 0,01]` cho MAE

chênh nhau chỉ **0,09**; OLS (`alpha = 0`) chỉ tốt hơn alpha đã đóng băng **0,07** (§8.1). Vậy hiệu năng gần như đến từ hồi quy tuyến tính, không phải từ L2. Nhóm giữ `alpha = 0,001` vì FINAL TEST đã bị xem — đổi bây giờ là test-informed model selection.

14.9 Cách dùng ở thời điểm phục vụ

Ứng dụng **chỉ nạp** artifact: không huấn luyện, không chọn lại alpha, không fit lại bộ tiền xử lý, không đọc mục tiêu từ dữ liệu. Mọi đặc trưng sinh bằng **cùng hàm** `src.features.build_features` với lúc huấn luyện → **không có train-serving skew** (có test đối chiếu từng cột trên dữ liệu thật). `is_holiday` tính từ lịch tất định; nhiệt độ nhận theo °C và tự quy đổi sang Kelvin; thời tiết mã hoá multi-hot — người dùng không phải nhập bất kỳ đặc trưng kỹ thuật hoá nào.

Chính sách phục vụ (đã đóng băng): `max(0, dự báo thô)` — chốt trong `models/serving_policy.json` bởi `src/freeze_serving_policy.py`, **chỉ dùng TRAIN + VALIDATION**; FINAL TEST 2018 **không** tham gia quyết định.

Căn cứ và điều kiện D1–D3 ở §12.5.3. API trả kèm `raw_model_output`, `predictor.deployed_prediction` và cờ `clipped_to_zero`; nếu một điều kiện không đạt thì policy tự đặt thành `"none"` và negative prediction thành limitation.

14.10 Quan hệ với FAIR / minh bạch

• **Rõ mục đích dùng** §14.6 · **rõ ngoài phạm vi** §14.7 · **rõ hạn chế** §14.8.

• **Số liệu lấy từ artifact, không gõ tay** — toàn bộ; có test kiểm tra frontend không hard-code.

• **Đo lường được bằng máy** — 392 test (`py -m pytest tests\ -v`), số test tự đối chiếu bằng `pytest --collect-only`.

• **Người dùng biết khi nào mô hình không đáng tin** — cảnh báo `in_dataset_range`, `state_fair_calendar_unknown` trong mọi response.

15. Hạn chế và ngoài phạm vi sử dụng

Danh sách đầy đủ 13 hạn chế nằm ở **§14.8** (Model Card). Nếu phải nói trong 30 giây, nhóm chọn 5 điểm có **hệ quả với quyết định của người dùng**: **phạm vi địa lý** — một trạm đo, không đại diện toàn thành phố; **ngày lễ** — MAE gấp 4,3 lần ngày thường, trên mẫu chỉ 7 ngày lịch; **giờ đêm** — baseline thắng Ridge ở 5/5 giờ 00–04 (§10.7); **phạm vi thời gian** — test 2018 chỉ 9 tháng đầu năm, và đó là phần *dễ hơn* của năm; và **an toàn** — không dùng cho mục đích safety-critical. Ngoài ra, vì đầu vào là **thời tiết quan sát** (không phải thời tiết dự báo), sai số thời tiết dự báo sẽ **cộng thêm** vào sai số mô hình khi vận hành thật (§14.8, hạn chế 9).

16. Kết luận và hướng mở rộng

16.1 Kết luận

• **Về phương pháp:** xây dựng được quy trình đánh giá trung thực trên dữ liệu chuỗi thời gian — tiền xử lý tất định, bộ tiền xử lý fit trên TRAIN duy nhất, time split có assert chặn, và FINAL TEST được bảo vệ bằng mã nguồn chứ không chỉ bằng lời hứa.

• **Về kết quả:** Ridge (alpha = 0,001) đạt **MAE 259,73 / RMSE 416,78 / R² 0,9554** trên FINAL TEST 2018, **nhỉnh hơn baseline 13,16 MAE (4,8 %)** trên cùng một tập dữ liệu — kèm điều kiện ở §9.3: hiệu ưu thế này **không nhất quán** qua các cửa sổ 2012–2017.

• **Về hiểu biết:** khoảng cách thời gian giữa tập huấn luyện và tập dự báo **làm thay đổi đáng kể **con số đánh giá (Thí nghiệm 1b/1c: chênh 4,15 MAE). Nhóm** không** khẳng định cơ chế là "mô hình nhìn thấy hàng xóm": phần lớn hiệu ứng không do kích thước tập huấn luyện, và mô hình không có đặc trưng lag nên không thể nhớ giá trị dòng lân cận. Đây là bài học trung tâm.

• **Về minh bạch và sản phẩm:** chỉ ra được mô hình hỏng ở đâu (ngày lễ, tuyết, giờ đêm) thay vì chỉ trích chỉ số tổng; web/API chạy được, chỉ nạp artifact, 392 test pass.

16.2 Hướng mở rộng

Hướng	Lý do	Cảnh báo về rò rỉ
Thêm đặc trưng lag của `traffic_volume`	Thí nghiệm 8 đo được: MAE time split giảm từ 298,74 xuống 172,66 (≈ −42 %) khi lag dựng **đúng theo thời điểm**	Bắt buộc ghép theo `date_time − k giờ`, **không** `shift()` theo dòng (cách sai làm tăng gấp đôi mức lạc quan); tuyệt đối không dùng lag nằm trong tương lai; cần backtest nhiều kỳ và kiểm thử train-serving skew. **Chưa đưa vào mô hình chính** vì FINAL TEST đã bị xem
Mô hình phi tuyến (LightGBM, gradient boosting)	Nhiều khả năng giảm MAE ở giờ đêm và ngày lễ	Phải giữ nguyên time split và quy tắc fit TRAIN only; dễ rơi vào bẫy chọn mô hình theo test
Tương tác `giờ × thời tiết`	Khớp trực tiếp với giả thuyết ở §10.7 về hiệu ứng tương đối ở giờ đêm	Phải chọn trên validation; cẩn thận với mẫu nhỏ ở giờ đêm
Mô hình riêng cho ngày lễ	MAE ngày lễ cao gấp 4,3 lần	Với chỉ 7 ngày lễ trong 2018, rất dễ overfit — cần dữ liệu nhiều hơn

16.3 Bài học

Trên cùng một bộ dữ liệu, **cách đánh giá** đã làm MAE chênh nhau đáng kể: random split lạc quan 5,43 ± 0,81 điểm so với time split **trên cùng một tập dòng đánh giá** (§11.1), và khoảng cách thời gian của tập huấn luyện làm thay đổi thêm 4,15 điểm (§11.3) — cả hai đều nhỏ với cải thiện 13,17 điểm mà mô hình đạt được so với baseline. Bài học quan trọng nhất: **đừng gán một cơ chế cho một con số khi chưa tách được các yếu tố.** Bản báo cáo cũ quy

6,55 MAE cho "mô hình nhìn thấy hàng xóm"; sau khi thêm arm đối chứng cùng kích thước, nhóm thấy phần lớn hiệu ứng **không** đến từ kích thước tập huấn luyện, và cơ chế "nhớ hàng xóm" **không khả thi** với mô hình không có đặc trưng lag. Con số giữ nguyên; **kết luận** phải sửa.

17. Phụ lục

17.1 Cấu trúc thư mục

```
data/ dữ liệu thô + đã làm sạch + audit + data dictionary
src/ download_data · data · features/holidays/weather · eda · train ·
experiments · freeze_serving_policy · evaluate ·
postprocess_audit · uncertainty_audit
app/ FastAPI + 3 màn hình web (chỉ load artifact)
models/ ridge_pipeline.joblib + metadata + baseline + serving_policy
reports/ final_report.md (file này) + figures/*.md, *.json, *.png
docs/ project-log.md, slides-outline.md, demo-script.md, viva-questions.md
release/ final_report.docx, final_report.pdf, slides.pptx (sinh tự động)
tests/ 392 test
```

17.2 Lệnh tái lập toàn bộ (Windows)

```
py -m pip install -r requirements-lock.txt
py src\download_data.py
py src\data.py
py src\eda.py
py src\train.py
py src\experiments.py
py src\freeze_serving_policy.py
py src\evaluate.py
py src\postprocess_audit.py
py src\uncertainty_audit.py
py -m pip install -r requirements-export.txt
py src\export_docs.py all
py -m pytest tests\ -v
py -m uvicorn app.main:app --reload
```

Vì sao `freeze_serving_policy.py` phải đứng trước `evaluate.py` **— lý do đầy đủ ở §12.5.2; hai script audit chỉ đo **hậu quả sau **khi policy đã đóng băng.** Kiểm chứng trên máy sạch:** `py src\freeze_serving_policy.py --check` — chỉ tính lại rồi so nội dung, **không ghi file nào**; exit 0 = khớp, exit 1 = lệch kèm diff (§12.5.2b).

Nếu chỉ muốn cài theo khoảng version tương thích thay vì tái lập tuyệt đối, dùng `py -m pip install -r requirements.txt`. Phiên bản thật của môi trường đã sinh artifact nằm trong `models/environment.json`. Seed cố định: `42`.

17.3 Danh mục tài liệu tham khảo

Tài liệu	Nội dung
`data/README.md` · `data/data_dictionary.md`	Nguồn, giấy phép, checksum, audit; mô tả biến và các bẫy đã xác minh
`reports/figures/data_quality_report.md` · `eda_train_only.md`	Chất lượng dữ liệu; EDA chỉ trên TRAIN
`reports/figures/experiments_report.md`	Thí nghiệm phát triển 2012–2017 (mục 1 → 8)
`reports/figures/evaluation_report.md`	FINAL TEST 2018 đầy đủ (24 giờ, 7 ngày, 11 loại thời tiết)
`reports/figures/postprocess_audit.md` · `uncertainty_audit.md`	Kiểm toán policy `max(0, ·)`; độ bất định bootstrap theo ngày, MAE theo tháng/giờ/ngày lễ
`reports/figures/alpha_sensitivity.json`	Độ nhạy alpha trên lưới mở rộng (chỉ VALIDATION)
`docs/project-log.md` · `docs/slides-outline.md` · `docs/demo-script.md` · `docs/viva-questions.md`	Nhật ký theo tuần (cần điền tên + giờ); dàn ý slide; kịch bản demo; 35 câu hỏi + đáp án
`requirements*.txt` · `models/environment.json`	Khoảng version, phiên bản ghim chính xác, công cụ xuất; môi trường + seed đọc được bằng máy

17.4 Nhật ký phát triển

Phần tóm tắt; bảng nhật ký đầy đủ (thời gian · người thực hiện · giờ ước lượng · công việc · kết quả · vấn đề) nằm ở **`docs/project-log.md`**.

Tuần	Việc	Kết quả
1	Brief, data README, dictionary, kế hoạch baseline	Chốt 7 quy tắc chống rò rỉ **trước khi** động vào dữ liệu

Tuần	Việc	Kết quả
2	Tải và làm sạch, audit, EDA trên TRAIN, time split	Phát hiện 5.445 nhóm trùng, 10 dòng `temp` vô lý, 1 dòng `rain_1h` sentinel
3	Baseline + Ridge pipeline + tune alpha	`alpha = 0,001`; vượt baseline ngay trên validation
4	Thí nghiệm 1, 1b, 1c, 3, 3b, 8 (chỉ 2012–2017)	Tách được kích thước / mức năm / khoảng cách thời gian; đo lạc quan 5,43 MAE
5	Đóng băng serving policy → FINAL TEST 2018 + phân tích lỗi	Policy chốt trên TRAIN+VAL; MAE 259,73; điểm yếu ở ngày lễ, tuyết, giờ đêm
6	FastAPI + 3 màn hình + 392 test + tài liệu + bản phát hành	Web/API chạy thật, không train-serving skew; bootstrap bất định + kiểm toán alpha
7	Đóng băng chạy lại được (`--check` / `--force`)	`serving_policy.json` giữ nguyên hash khi chạy lại

17.5 Tài liệu phát hành

Báo cáo này được soạn bằng Markdown để **kiểm chứng được đối chiếu artifact** — mọi con số đều do `tests/test_report.py` đối chiếu với `models/` và `reports/figures/`. Bản nộp được **sinh tự động từ chính file Markdown đó**, không chép số tay.

```
py -m pip install -r requirements-export.txt
py src\export_docs.py all :: sinh cả 3 định dạng
py src\export_docs.py docx :: hoặc từng định dạng
py src\export_docs.py pdf
py src\export_docs.py pptx
```

Tệp	Nguồn	Công cụ	Quy mô
`release/final_report.docx`	`reports/final_report.md`	pandoc (pypandoc-binary)	17 mục, 58 bảng, 7 ảnh nhúng
`release/final_report.pdf`	`reports/final_report.md`	reportlab + font Arial	**25 trang A4**
`release/slides.pptx`	`docs/slides-outline.md`	python-pptx	11 slide, 4 ảnh thật

25 trang A4 nằm trong khoảng mục tiêu 15–25 trang. Nếu thiếu một công cụ, script in "BỎ QUA" kèm lý do và **không** tạo file rỗng — để không ai tưởng đã xuất xong.

Tài liệu đi kèm giữ nguyên dạng Markdown vì nhóm còn phải điền: `docs/project-log.md` (nhật ký theo tuần — **cần người dùng điền** tên + giờ thật), `docs/slides-outline.md` (dàn ý 11 slide), `docs/demo-script.md` (kịch bản demo 6 phút 40 giây) và `docs/viva-questions.md` (35 câu hỏi + đáp án, và 10 câu bổ sung).

17.6 Nhóm & phân công

□ CẦN NGƯỜI DÙNG ĐIỀN TRƯỚC KHI NỘP Mục này **cố ý để trống có marker**. Nhóm **không** tự bịa tên, không bịa phân công, không bịa số giờ làm. Ba dòng dưới phải do chính thành viên điền bằng thông tin thật:

Trường	Cần điền
Thành viên 1	`[TÊN THÀNH VIÊN 1]`
Thành viên 2	`[TÊN THÀNH VIÊN 2]`
Phân công thực tế theo tuần	`[PHÂN CÔNG THỰC TẾ]` — theo khuôn `docs/project-log.md`

Phần **công việc kỹ thuật theo tuần đã có sẵn** trong `docs/project-log.md`; chỉ cần bổ sung cột *Người thực hiện* và *Giờ ước lượng* bằng dữ liệu thật. **Về lịch sử Git:** hiện lịch sử commit thuộc một tài khoản duy nhất. Nếu đề yêu cầu mỗi thành viên có phần đóng góp riêng, phần đó **phải do chính thành viên đó tạo** — không đổi tên tác giả, không tạo commit giả cho người khác, không backdate commit.

17.7 Công cụ AI đã sử dụng

Công cụ AI đã sử dụng:

• ChatGPT

• Pi Agent

Mục đích sử dụng: phân tích yêu cầu đề bài; rà soát phương pháp chống data leakage; hỗ trợ viết/sửa mã nguồn và xây dựng test; kiểm tra Web/API; chuẩn bị báo cáo, slide và câu hỏi vấn đáp.

Cách kiểm chứng lại: chạy pipeline trên dataset UCI gốc và đối chiếu SHA256 (`data/README.md`); chạy `py -m pytest tests\ -v`; đối chiếu mọi metric trong báo cáo với artifact do mã nguồn sinh ra (`models/*.json`, `reports/figures/*.json`); smoke-test FastAPI/Web kể cả các ca nhập sai; đọc lại từng dòng mã nguồn và tài liệu trước khi bảo vệ.

Ranh giới nhóm tự đặt: AI **không** quyết định giá trị nghiệm vụ (quy tắc lịch ngày lễ, định nghĩa danh mục thời tiết, ngưỡng giá trị vô lý) và **không** tạo ra số liệu. Mọi con số trong báo cáo đến từ artifact do mã nguồn sinh ra;

`tests/test_report.py` tự động đối chiếu tài liệu với artifact, nên tài liệu không thể lệch số mà không bị test bắt.